Predicción de etiquetas algorítmicas y dificultad de problemas de Codeforces

Ajuste fino multitarea de ModernBERT

Open in Colab

Instrucciones de ejecución

1. Abrir el notebook en Google Colab y seleccionar *Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU T4*.
2. (Opcional) Añadir el token de Hugging Face como secreto de Colab con el nombre `HF_TOKEN` (icono de la llave de la barra lateral). Sin token el notebook funciona igual, pero no sube nada al Hub.
3. Ejecutar *Entorno de ejecución → Ejecutar todo*.

Dos *flags* de la celda de configuración controlan la ejecución:

Flag	Efecto
`RUN_TRAINING`	`True` : entrena el modelo. `False` : lo descarga ya ajustado desde el Hub.
`PUSH_TO_HUB`	Sube el modelo y el dataset procesado al Hub (requiere token).

El modelo se sube al Hub al final de cada época, de modo que una desconexión de Colab no obliga a repetir todo el entrenamiento.

Tiempo aproximado en una T4: el ajuste fino tarda unos 35 minutos (35,5 min en la ejecución de este notebook) y la extracción de *embeddings* del *linear probe*, algo más de 2 minutos; el resto de celdas añade unos pocos minutos.

1. Datos del alumno y título del proyecto

- **Alumno:** Alex Sierra Alcalá
- **Asignatura:** Deep Learning for NLP — Máster Universitario en Inteligencia Artificial
- **Proyecto:** *Predicción de etiquetas algorítmicas y dificultad de problemas de Codeforces mediante ajuste fino multitarea de ModernBERT*
- **Modelo ajustado en el Hub:** AlexSA45/modernbert-codeforces-tags-rating

2. Introducción

2.1. La tarea

Codeforces es una de las plataformas de programación competitiva más usadas. Cada problema lleva asociadas dos piezas de metadatos:

- **Etiquetas algorítmicas** (`dp` , `greedy` , `graphs` , `math` ...): las técnicas con las que se puede resolver. Un problema suele tener varias, así que es un problema de **clasificación multi-etiqueta**.
- **Rating de dificultad** (de 800 a 3500): estima la habilidad necesaria para resolverlo. Es un problema de **regresión**.

En este proyecto se predicen **ambas cosas a la vez** a partir únicamente del enunciado en inglés. Un sistema así sirve para recomendar problemas con los que entrenar una técnica concreta, estimar la dificultad de un problema nuevo o etiquetar colecciones de problemas sin metadatos.

La tarea es difícil porque el enunciado describe *qué* hay que calcular, no *cómo*. Además, los números de las restricciones (por ejemplo, $n \leq 2 \cdot 10^5$ frente a $n \leq 20$) son pistas decisivas sobre la complejidad esperada de la solución.

2.2. Por qué ModernBERT

Se usa `answerdotai/ModernBERT-base` (Warner et al., 2024), un *encoder* bidireccional al estilo BERT con un diseño moderno. Según su *model card*:

- Está preentrenado con **2 billones de tokens de texto en inglés y código**, un dominio cercano al de los enunciados de programación competitiva.
- Tiene un **contexto nativo de 8192 tokens**, frente a los 512 de BERT. Muchos enunciados de Codeforces superan los 512 tokens.
- Tiene 22 capas y **149 millones de parámetros**.

Un *encoder* encaja mejor que un LLM generativo en esta tarea: la salida es un vector de probabilidades y un número, no texto.

Con unos 150M de parámetros, el modelo, sus gradientes y los estados del optimizador caben en los 16 GB de una T4. Técnicas como LoRA o QLoRA están pensadas para modelos de miles de millones de parámetros que no caben en memoria; aquí no son necesarias, y el ajuste fino completo permite adaptar todas las capas a un dominio muy técnico.

2.3. Por qué multitarea

Etiquetas y dificultad están relacionadas: algunas técnicas aparecen sobre todo en problemas difíciles y otras en problemas fáciles. Compartir el *encoder* entre las dos tareas permite aprovechar esa relación, y se obtiene un único modelo que resuelve ambas con una sola pasada.

2.4. Preparación del entorno

Instalamos las librerías necesarias. Se fijan las versiones del ecosistema Hugging Face con las que se ha probado el notebook.

- `transformers` : modelos, tokenizadores y la clase `Trainer` .
- `datasets` : contenedor `Dataset` usado por el `Trainer` .
- `accelerate` : abstracción del hardware usada internamente por el `Trainer` .
- `huggingface_hub` : lectura del dataset y subida del modelo al Hub.
- `scikit-learn` : baselines clásicos y métricas.

In [22]:
```
%pip install -q transformers==5.19.0 datasets==5.1.0 accelerate==1.15.0
%pip install -q huggingface_hub==1.33.0 scikit-learn pyarrow seaborn
```

Importaciones, versiones y semillas. `transformers.set_seed` fija la semilla de `random` , `numpy` y `torch` para que los resultados sean reproducibles.

In [23]:
```
import collections
import hashlib
import json
import os
import re
import time
import warnings
from dataclasses import dataclass
from typing import Optional

import datasets
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pyarrow.parquet as pq
import seaborn as sns
import sklearn
import torch
import torch.nn as nn
import torch.nn.functional as F
import transformers
from datasets import Dataset
from huggingface_hub import HfApi, HfFileSystem, hf_hub_download, login
from IPython.display import display
from scipy.stats import spearmanr
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import average_precision_score, f1_score, precision_recall_fscore_support
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader
from transformers import (
    AutoConfig,
    AutoModel,
    AutoTokenizer,
    EarlyStoppingCallback,
    PretrainedConfig,
    PreTrainedModel,
    Trainer,
    TrainingArguments,
    set_seed,
)
from transformers.utils import ModelOutput

warnings.filterwarnings("ignore")
transformers.logging.set_verbosity_error()
datasets.disable_progress_bars()
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["WANDB_DISABLED"] = "true"

for lib in [torch, transformers, datasets, sklearn, np, pd]:
    print(f"{lib.__name__:>14}: {lib.__version__}")

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
if torch.cuda.is_available():
    print(f"GPU detectada: {torch.cuda.get_device_name(0)}")
else:
    print("No se ha detectado GPU")
```

```
         torch: 2.14.1+cu130
  transformers: 5.19.0
      datasets: 5.1.0
       sklearn: 1.6.1
         numpy: 2.4.6
        pandas: 2.2.3
GPU detectada: Tesla T4
```

2.5. Configuración

Todos los hiperparámetros y *flags* están en esta celda.

In [24]:
```
CONFIG = {
    # Flags de ejecución
    "RUN_TRAINING": True,
    "PUSH_TO_HUB": True,
    # Hub
    "HUB_MODEL_ID": "AlexSA45/modernbert-codeforces-tags-rating",
    "HUB_DATASET_ID": "AlexSA45/codeforces-tags-rating-processed",
    # Datos
    "DATASET_ID": "open-r1/codeforces",
    "MIN_TAG_COUNT": 100,     # mínimo de apariciones en train para conservar una etiqueta
    "TOP_K_TAGS": 20,         # máximo de etiquetas a conservar
    # Modelo
    "BASE_MODEL": "answerdotai/ModernBERT-base",
    "MAX_LEN": 1024,
    "DROPOUT": 0.1,
    "LAMBDA_RATING": 0.5,
    "POS_WEIGHT_CLIP": 10.0,
    # Entrenamiento
    "LR": 5e-5,
    "EPOCHS": 5,
    "BATCH_SIZE": 8,
    "GRAD_ACCUM": 2,
    "WARMUP_RATIO": 0.1,
    "WEIGHT_DECAY": 0.01,
    "EARLY_STOPPING_PATIENCE": 2,
    "SEED": 42,
}

MAX_STEPS = -1

SEED = CONFIG["SEED"]
```

```
set_seed(SEED)
print(json.dumps(CONFIG, indent=1))
```


```
{
 "RUN_TRAINING": true,
 "PUSH_TO_HUB": true,
 "HUB_MODEL_ID": "AlexSA45/modernbert-codeforces-tags-rating",
 "HUB_DATASET_ID": "AlexSA45/codeforces-tags-rating-processed",
 "DATASET_ID": "open-r1/codeforces",
 "MIN_TAG_COUNT": 100,
 "TOP_K_TAGS": 20,
 "BASE_MODEL": "answerdotai/ModernBERT-base",
 "MAX_LEN": 1024,
 "DROPOUT": 0.1,
 "LAMBDA_RATING": 0.5,
 "POS_WEIGHT_CLIP": 10.0,
 "LR": 5e-05,
 "EPOCHS": 5,
 "BATCH_SIZE": 8,
 "GRAD_ACCUM": 2,
 "WARMUP_RATIO": 0.1,
 "WEIGHT_DECAY": 0.01,
 "EARLY_STOPPING_PATIENCE": 2,
 "SEED": 42
}
```


2.7. Autenticación en Hugging Face Hub

In [25]:
```
HF_TOKEN = os.environ.get("HF_TOKEN")
try:
    from google.colab import userdata
    HF_TOKEN = userdata.get("HF_TOKEN")
except Exception:
    pass

if HF_TOKEN:
    login(token=HF_TOKEN)
    print("Sesión iniciada en el Hub.")
else:
    CONFIG["PUSH_TO_HUB"] = False
    print("Sin token: no se subirá nada al Hub (PUSH_TO_HUB=False).")
PUSH_TO_HUB = CONFIG["PUSH_TO_HUB"]
```

```
Sesión iniciada en el Hub.
```

3. Dataset

3.1. Fuente

Se usa `open-r1/codeforces`, publicado por Hugging Face dentro del proyecto Open-R1. Contiene algo más de 10 000 problemas únicos de Codeforces, desde 2010 hasta principios de 2025, con el enunciado dividido en campos, sus etiquetas y su rating.

Campos usados y descartados.

Campo	Uso	Motivo
`title`, `description`, `input_format`, `output_format`, `interaction_format`, `note`	Entrada	Enunciado del problema
`tags`	Objetivo 1	Etiquetas algorítmicas
`rating`	Objetivo 2	Dificultad
`id`, `aliases`, `contest_id`, `contest_start`, `index`	Auxiliar	Deduplicación y división temporal
`editorial`	**Descartado**	Explica la solución: sería una fuga de información
`examples`	Descartado	Casos numéricos con poca señal y muchos tokens
`official_tests`, `generated_checker`	Descartado	Tests y validadores, sin relación con la tarea

Lectura eficiente. Los ficheros parquet del dataset pesan unos 2,7 GB, casi todo por la columna `official_tests`. Como parquet guarda los datos por columnas, leemos solo las que necesitamos con `HfFileSystem` y `pyarrow` y descargamos apenas unos MB. Así, además, el campo `editorial` ni siquiera llega a descargarse.

In [26]:
```
COLUMNS = [
    "id", "aliases", "contest_id", "contest_name", "contest_start", "index", "title",
    "description", "input_format", "output_format", "interaction_format", "note", "rating",
    "tags",
]

fs = HfFileSystem()
parquet_files = sorted(fs.glob(f"datasets/{CONFIG['DATASET_ID']}/data/*.parquet"))
frames = []
t0 = time.time()
for path in parquet_files:
    with fs.open(path, "rb", block_size=2**20) as fh:
        frames.append(pq.read_table(fh, columns=COLUMNS).to_pandas())
raw = pd.concat(frames, ignore_index=True)
print(f"{len(parquet_files)} ficheros, {len(raw)} problemas leídos en {time.time() - t0:.0f} s")
raw.head(3)
```

```
12 ficheros, 10024 problemas leídos en 9 s
```

Out[26]:

	id	aliases	contest_id	contest_name	contest_start	index	title	description	input_format	output_format	interaction_format	note	rating	tags
0	2063/A	None	2063	Codeforces Round 1000 (Div. 2)	1737547500	A	Minimal Coprime	Today, Little John used all his savings to buy...	Each test contains multiple test cases. The fi...	For each test case, output the number of minim...	None	On the first test case, the given segment is $...	800.0	[math, number theory]
1	2063/B	None	2063	Codeforces Round 1000 (Div. 2)	1737547500	B	Subsequence Update	After Little John borrowed expansion screws fr...	Each test contains multiple test cases. The fi...	For each test case, output the minimum value o...	None	On the second test case, the array is $$$a=[1,...	1100.0	[constructive algorithms, data structures, gre...
2	2063/C	None	2063	Codeforces Round 1000 (Div. 2)	1737547500	C	Remove Exactly Two	Recently, Little John got a tree from his aunt...	Each test contains multiple test cases. The fi...	For each test case, output the maximum number ...	None	On the first test case, removing a vertex twic...	1600.0	[brute force, data structures, dfs and similar...

3.2. Construcción del texto de entrada

Unimos todos los *splits* originales y construimos un único texto por problema, en este orden:

```
Title: ...
Input: ...
Output: ...
Interaction: ...   (solo problemas interactivos)
Statement: ...
Note: ...
```

Las restricciones (por ejemplo, $1 \le n \le 2 \cdot 10^5$) están casi siempre en la sección de entrada y son la pista más informativa sobre la complejidad de la solución. Al ponerlas al principio, si el texto supera `MAX_LEN` tokens se trunca la narrativa del final y no ellas.

Limpieza de LaTeX. Codeforces escribe las fórmulas entre `$$$`. Sustituimos los comandos más frecuentes por su equivalente legible (`\le` → `<=`, `10^{5}` → `10^5`, `\texttt{x}` → `x` ...) y **conservamos siempre los números**, porque forman parte de la señal.

In [27]:
```
LATEX_RULES = [
    (r"\$\$\$", ""),
    (r"\^\{\\text\{[^{}]*\}\}", ""),                    # marcas de nota al pie: ^{\text{*}}
    (r"\\(?:leq|le)(?![a-zA-Z])", "<="),
    (r"\\(?:geq|ge)(?![a-zA-Z])", ">="),
    (r"\\(?:neq|ne)(?![a-zA-Z])", "!="),
    (r"\\(?:cdot|times)(?![a-zA-Z])", "*"),
    (r"\\(?:ldots|dots|cdots)(?![a-zA-Z])", "..."),
    (r"\\(?:texttt|textbf|textit|text|mathit|mathrm|operatorname)\{([^{}]*)\}", r"\1"),
    (r"\^\{([^{}]*)\}", r"^\1"),                        # 10^{5} -> 10^5
    (r"\\([a-zA-Z]+)", r"\1"),                          # resto: \gcd -> gcd, \max -> max
    (r"\s+", " "),                                      # colapsar espacios
]
LATEX_RULES = [(re.compile(p), r) for p, r in LATEX_RULES]


def clean_latex(text):
    """Limpia el LaTeX de Codeforces conservando números y operadores."""
    if not isinstance(text, str):
        return ""
    for pattern, repl in LATEX_RULES:
        text = pattern.sub(repl, text)
```

```
    return text.strip()


def build_text(row):
    """Concatena los campos del enunciado en el orden descrito arriba."""
    parts = [
        ("Title", row["title"]),
        ("Input", row["input_format"]),
        ("Output", row["output_format"]),
        ("Interaction", row["interaction_format"]),
        ("Statement", row["description"]),
        ("Note", row["note"]),
    ]
    lines = []
    for name, value in parts:
        value = clean_latex(value)
        if value:
            lines.append(f"{name}: {value}")
    return "\n".join(lines)


raw["text"] = raw.apply(build_text, axis=1)

example = raw[raw["input_format"].fillna("").str.contains(r"\\le")].iloc[0]
print("ANTES (input_format):\n", example["input_format"][:400])
print("\nDESPUÉS:\n", clean_latex(example["input_format"])[:400])
print("\nTEXTO COMPLETO (primeros 700 caracteres):\n", example["text"][:700])
```

```
ANTES (input_format):
 Each test contains multiple test cases. The first line contains the number of test cases $$$t$$$ ($$$1 \le t \le 100$$$). The description of the test cases follows.

The only line of each test case consists of two integers $$$l$$$ and $$$r$$$ ($$$1 \le l \le r \le 10^9$$$).

DESPUÉS:
 Each test contains multiple test cases. The first line contains the number of test cases t (1 <= t <= 100). The description of the test cases follows. The only line of each test case consists of two integers l and r (1 <= l <= r <= 10^9).

TEXTO COMPLETO (primeros 700 caracteres):
 Title: Minimal Coprime
Input: Each test contains multiple test cases. The first line contains the number of test cases t (1 <= t <= 100). The description of the test cases follows. The only line of each test case consists of two integers l and r (1 <= l <= r <= 10^9).
Output: For each test case, output the number of minimal coprime segments contained in [l,r], on a separate line.
Statement: Today, Little John used all his savings to buy a segment. He wants to build a house on this segment. A segment of positive integers [l,r] is called coprime if l and r are coprime. A coprime segment [l,r] is called minimal coprime if it does not contain any coprime segment not equal to itself. To better un
```

3.3. Limpieza y deduplicación

- Se eliminan los problemas **sin enunciado**.
- **Duplicados por alias:** los problemas compartidos entre la Div. 1 y la Div. 2 se registran en `aliases`. Se agrupan con una clave común (el menor identificador del grupo) y se conserva la aparición más antigua.
- **Duplicados por texto:** se calcula un *hash* del texto normalizado (minúsculas y sin espacios) y también se conserva solo la aparición más antigua.

Si un mismo enunciado apareciera en train y en test, el modelo podría memorizarlo y la evaluación sería demasiado optimista.

In [28]:
```
df = raw[raw["description"].fillna("").str.strip() != ""].copy()
n_no_desc = len(raw) - len(df)
df = df.sort_values(["contest_start", "contest_id", "index"]).reset_index(drop=True)

all_ids = set(df["id"])


def canonical_id(row):
    aliases = row["aliases"] if row["aliases"] is not None else []
    return min([row["id"]] + [a for a in aliases if a in all_ids])
```

```
df["canonical_id"] = df.apply(canonical_id, axis=1)
n_before = len(df)
df = df.drop_duplicates("canonical_id", keep="first")
n_alias_dups = n_before - len(df)

df["text_hash"] = df["text"].str.lower().str.replace(r"\s+", "", regex=True).map(
    lambda s: hashlib.md5(s.encode()).hexdigest())
n_before = len(df)
df = df.drop_duplicates("text_hash", keep="first").reset_index(drop=True)
n_text_dups = n_before - len(df)

# Las meta-etiquetas que empiezan por '*' (p. ej. '*special') no son técnicas algorítmicas
df["tags_clean"] = df["tags"].map(
    lambda ts: sorted({t for t in (ts if ts is not None else []) if not t.startswith("*")}))
df["date"] = pd.to_datetime(df["contest_start"], unit="s")

print(f"Problemas sin enunciado eliminados: {n_no_desc}")
print(f"Duplicados por alias eliminados:    {n_alias_dups}")
print(f"Duplicados por texto eliminados:    {n_text_dups}")
print(f"Problemas restantes:                {len(df)}")
```

```
Problemas sin enunciado eliminados: 43
Duplicados por alias eliminados:    0
Duplicados por texto eliminados:    5
Problemas restantes:                9976
```

3.4. División temporal

La división es por fecha del concurso, nunca aleatoria: el 80 % más antiguo para train, el 10 % siguiente para validación y el 10 % más reciente para test. Así se entrena con problemas antiguos y se evalúa con problemas nuevos, igual que ocurriría al usar el modelo en un concurso futuro. Una división aleatoria mezclaría problemas del mismo concurso, que comparten autores y estilo, entre train y test.

In [29]:
```
dates = df["date"].sort_values().reset_index(drop=True)
val_cut = dates.iloc[int(0.8 * len(df))]
test_cut = dates.iloc[int(0.9 * len(df))]
df["split"] = np.where(df["date"] < val_cut, "train",
                       np.where(df["date"] < test_cut, "val", "test"))

assert (df.groupby("contest_id")["split"].nunique() == 1).all()
print("Corte val :", val_cut)
print("Corte test:", test_cut)
```

```
Corte val : 2022-10-10 14:35:00
Corte test: 2023-12-11 14:35:00
```

3.5. Vocabulario de etiquetas y normalización del rating

Todo lo que se aprende de los datos se calcula solo con train:

- **Vocabulario:** etiquetas con al menos `MIN_TAG_COUNT` apariciones en train, hasta un máximo de `TOP_K_TAGS`. Las etiquetas muy raras (`2-sat`, `schedules`...) tienen demasiado pocos ejemplos para aprenderlas. Se descartan los problemas que se quedan sin ninguna etiqueta del vocabulario.
- **Rating:** se normaliza con *z-score* usando la media y la desviación típica de train. Los problemas sin rating se conservan con `rating_mask = 0`, de modo que no cuentan ni en la pérdida ni en las métricas de rating.

In [30]:
```
train_tag_counts = collections.Counter(
    t for ts in df.loc[df["split"] == "train", "tags_clean"] for t in ts)
TAGS = [t for t, c in train_tag_counts.most_common()
        if c >= CONFIG["MIN_TAG_COUNT"]][:CONFIG["TOP_K_TAGS"]]
TAG2IDX = {t: i for i, t in enumerate(TAGS)}
N_TAGS = len(TAGS)
print(f"{N_TAGS} etiquetas conservadas:", TAGS)

df["tags_kept"] = df["tags_clean"].map(lambda ts: [t for t in ts if t in TAG2IDX])
no_tags = df["tags_kept"].map(len) == 0
print("Problemas sin ninguna etiqueta conservada:",
      no_tags.groupby(df["split"]).sum().to_dict())
df = df[~no_tags].reset_index(drop=True)
```

```
def multi_hot(tags):
    y = np.zeros(N_TAGS, dtype=np.float32)
    y[[TAG2IDX[t] for t in tags]] = 1.0
    return y


df["y_tags"] = df["tags_kept"].map(multi_hot)
df["rating_mask"] = df["rating"].notna().astype(np.float32)
train_ratings = df.loc[(df["split"] == "train") & df["rating"].notna(), "rating"]
RATING_MEAN, RATING_STD = float(train_ratings.mean()), float(train_ratings.std())
df["rating_z"] = ((df["rating"] - RATING_MEAN) / RATING_STD).fillna(0.0).astype(np.float32)
print(f"Rating en train: media={RATING_MEAN:.1f}, desviación={RATING_STD:.1f}")

SPLITS = ["train", "val", "test"]
data = {s: df[df["split"] == s].reset_index(drop=True) for s in SPLITS}
```

```
20 etiquetas conservadas: ['implementation', 'math', 'greedy', 'dp', 'data structures',
'constructive algorithms', 'brute force', 'graphs', 'sortings', 'binary search', 'dfs and similar',
'trees', 'strings', 'number theory', 'combinatorics', 'bitmasks', 'two pointers', 'geometry',
'dsu', 'shortest paths']
Problemas sin ninguna etiqueta conservada: {'test': 22, 'train': 395, 'val': 14}
Rating en train: media=1856.8, desviación=698.1
```

3.6. Análisis exploratorio

Tamaño y rango de fechas de cada split

In [31]:
```
summary = pd.DataFrame({
    s: {
        "problemas": len(d),
        "concursos": d["contest_id"].nunique(),
        "desde": d["date"].min().date(),
        "hasta": d["date"].max().date(),
        "con rating": int(d["rating_mask"].sum()),
        "rating medio": round(d["rating"].mean(), 1),
        "etiquetas/problema": round(d["tags_kept"].map(len).mean(), 2),
    } for s, d in data.items()
}).T
summary
```

Out[31]:

	problemas	concursos	desde	hasta	con rating	rating medio	etiquetas/problema
train	7585	1518	2010-02-19	2022-10-07	7461	1856.8	2.54
val	975	153	2022-10-10	2023-12-09	971	1790.4	3.15
test	985	139	2023-12-11	2025-02-11	977	1826.0	3.43

Definimos un estilo común para las figuras.

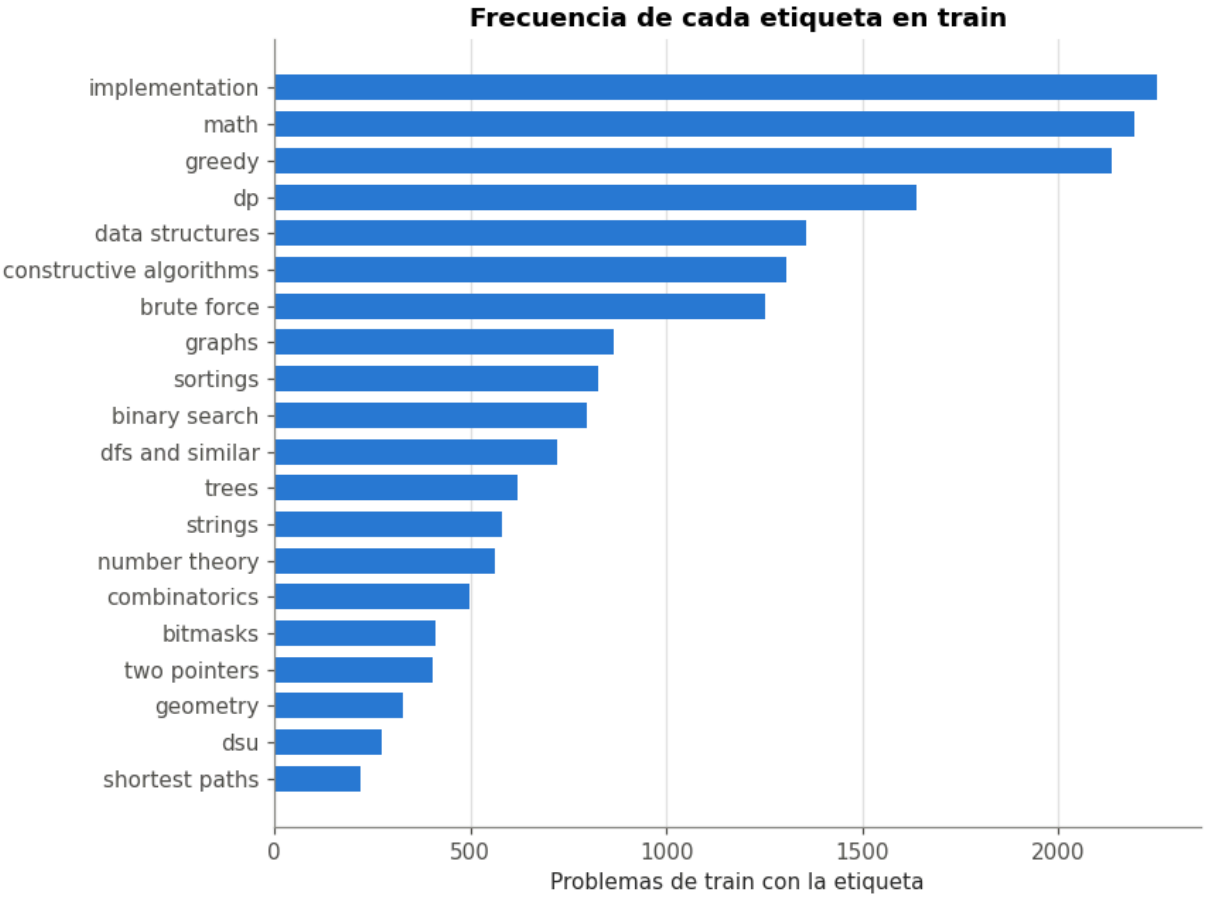
In [32]:
```
COLORS = {"train": "#2a78d6", "val": "#eb6834", "test": "#1baf7a"}
MODEL_COLORS = {"ModernBERT ajustado": "#2a78d6", "TF-IDF + LR": "#eb6834",
                "Linear probe": "#1baf7a"}
plt.rcParams.update({
    "figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": "#e5e5e3", "grid.linewidth": 0.8,
    "axes.edgecolor": "#8a8984", "axes.labelcolor": "#2b2b2a", "xtick.color": "#52514e",
    "ytick.color": "#52514e", "axes.titleweight": "bold", "axes.axisbelow": True,
})
```

Frecuencia de las etiquetas

In [33]:
```
tag_freq = pd.DataFrame({s: d["tags_kept"].explode().value_counts() for s, d in data.items()})
tag_freq = tag_freq.reindex(TAGS).fillna(0).astype(int)
for s in SPLITS:
    tag_freq[f"% {s}"] = (100 * tag_freq[s] / len(data[s])).round(1)

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(TAGS[::-1], tag_freq["train"][::-1], color=COLORS["train"], height=0.7)
ax.set_xlabel("Problemas de train con la etiqueta")
ax.set_title("Frecuencia de cada etiqueta en train")
ax.grid(axis="y", visible=False)
plt.tight_layout()
```

```
plt.show()
tag_freq
```



Out[33]:

tags_kept	train	val	test	% train	% val	% test
implementation	2252	229	288	29.7	23.5	29.2
math	2194	393	421	28.9	40.3	42.7
greedy	2137	388	463	28.2	39.8	47.0
dp	1639	255	282	21.6	26.2	28.6
data structures	1356	201	242	17.9	20.6	24.6
constructive algorithms	1308	241	234	17.2	24.7	23.8
brute force	1252	227	275	16.5	23.3	27.9
graphs	865	102	110	11.4	10.5	11.2
sortings	826	139	135	10.9	14.3	13.7
binary search	796	150	158	10.5	15.4	16.0
dfs and similar	723	106	116	9.5	10.9	11.8
trees	619	105	104	8.2	10.8	10.6
strings	579	78	77	7.6	8.0	7.8
number theory	563	101	105	7.4	10.4	10.7
combinatorics	498	94	109	6.6	9.6	11.1
bitmasks	412	84	90	5.4	8.6	9.1
two pointers	403	84	74	5.3	8.6	7.5
geometry	329	29	27	4.3	3.0	2.7
dsu	274	43	48	3.6	4.4	4.9
shortest paths	221	27	23	2.9	2.8	2.3

Coste del vocabulario

Quedarnos con las `TOP_K_TAGS` etiquetas más frecuentes simplifica el problema, pero tiene un coste: las etiquetas descartadas desaparecen y algunos problemas se quedan sin ninguna. Medimos ese coste en train y cuánto cubren las K etiquetas más frecuentes para distintos valores de K.

In [34]:

```
from matplotlib.ticker import PercentFormatter

total_occ = sum(train_tag_counts.values())
lost = {t: c for t, c in train_tag_counts.most_common() if t not in TAG2IDX}
n_train_before = int((dates < val_cut).sum())
n_train_dropped = n_train_before - len(data["train"])
keeps_all = (data["train"]["tags_clean"].map(len) == data["train"]["tags_kept"].map(len)).mean()

print(f"Etiquetas distintas en train: {len(train_tag_counts)}; conservadas: {N_TAGS}")
print(f"Apariciones de etiquetas perdidas: {sum(lost.values())} de {total_occ} "
```

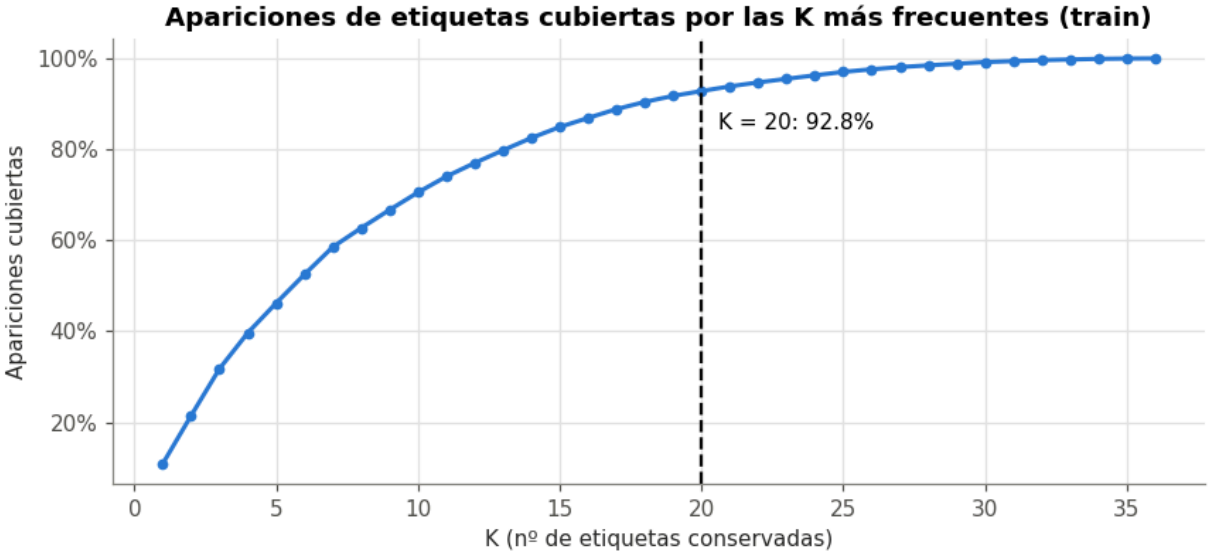
```
      f"({100 * sum(lost.values()) / total_occ:.1f}%)")
print(f"Problemas de train descartados (sin ninguna etiqueta conservada, incluidos los que "
      f"no tenían etiquetas): {n_train_dropped} de {n_train_before} "
      f"({100 * n_train_dropped / n_train_before:.1f}%)")
print(f"Problemas de train conservados con todas sus etiquetas: {100 * keeps_all:.1f}%")

counts_sorted = np.array([c for _, c in train_tag_counts.most_common()])
coverage = np.cumsum(counts_sorted) / counts_sorted.sum()
ks = np.arange(1, len(coverage) + 1)

fig, ax = plt.subplots(figsize=(8, 3.8))
ax.plot(ks, coverage, color=COLORS["train"], linewidth=2, marker="o", markersize=4)
ax.axvline(N_TAGS, color="#0b0b0b", linestyle="--", linewidth=1.5)
ax.annotate(f"K = {N_TAGS}: {100 * coverage[N_TAGS - 1]:.1f}%",
            (N_TAGS, coverage[N_TAGS - 1]), xytext=(8, -18), textcoords="offset points")
ax.yaxis.set_major_formatter(PercentFormatter(1))
ax.set(title="Apariciones de etiquetas cubiertas por las K más frecuentes (train)",
       xlabel="K (nº de etiquetas conservadas)", ylabel="Apariciones cubiertas")
plt.tight_layout()
plt.show()

pd.Series(lost, name="apariciones en train").to_frame().T
```

```
Etiquetas distintas en train: 36; conservadas: 20
Apariciones de etiquetas perdidas: 1488 de 20734 (7.2%)
Problemas de train descartados (sin ninguna etiqueta conservada, incluidos los que no tenían
etiquetas): 395 de 7980 (4.9%)
Problemas de train conservados con todas sus etiquetas: 84.0%
```



Out[34]:

	divide and conquer	probabilities	hashing	games	interactive	flows	matrices	string suffix structures	fft	graph matchings	ternary search	meet-in-the-middle	expression parsing	2-sat	chinese remainder theorem	schedules
apariciones en train	213	182	162	161	156	117	103	77	77	72	46	41	33	27	14	7

Coocurrencia de etiquetas

Medimos qué etiquetas aparecen juntas en train con el *lift*: $\mathrm{lift}(A,B) = P(A,B) \,/\, (P(A)\,P(B))$. Un *lift* de 1 indica que aparecen juntas lo esperable por azar; mayor que 1, que tienden a ir juntas, y menor que 1, que se evitan. Si el modelo aprende estas asociaciones, puede predecir pares de etiquetas aunque el texto solo dé pistas de una de ellas.

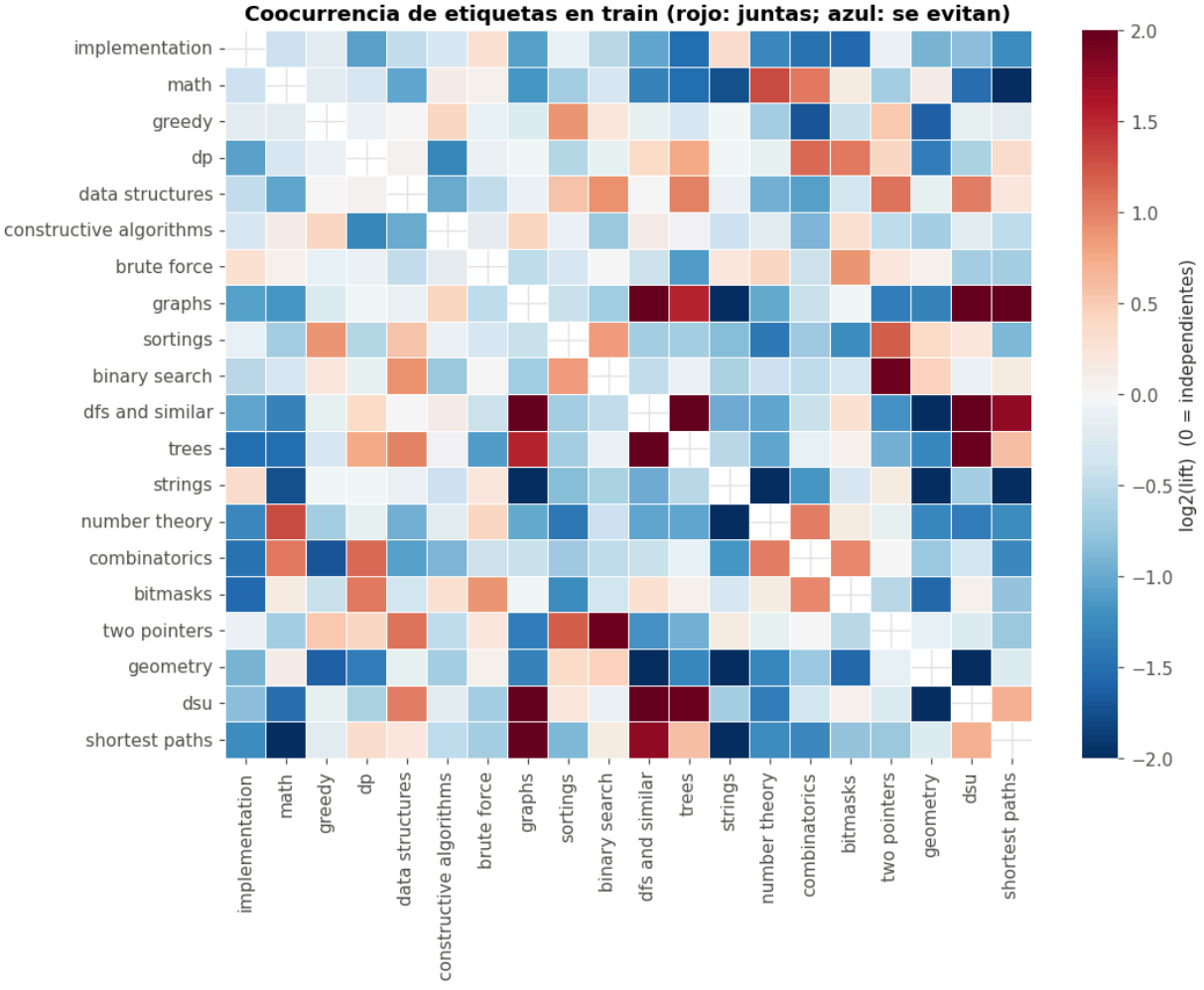
In [35]:

```
Y_tr = np.stack(data["train"]["y_tags"])
co = Y_tr.T @ Y_tr
p = Y_tr.mean(axis=0)
lift = (co / len(Y_tr)) / np.outer(p, p)
log_lift = np.log2(np.clip(lift, 2 ** -2, None))
np.fill_diagonal(log_lift, np.nan)

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(log_lift, xticklabels=TAGS, yticklabels=TAGS, cmap="RdBu_r", center=0,
            vmin=-2, vmax=2, linewidths=0.5, linecolor="white",
            cbar_kws={"label": "log2(lift)  (0 = independientes)"}, ax=ax)
ax.set_title("Coocurrencia de etiquetas en train (rojo: juntas; azul: se evitan)")
plt.tight_layout()
```

```
plt.show()

pairs = [(TAGS[i], TAGS[j], int(co[i, j]), lift[i, j])
         for i in range(N_TAGS) for j in range(i + 1, N_TAGS) if co[i, j] >= 30]
top_pairs = pd.DataFrame(pairs, columns=["etiqueta A", "etiqueta B", "problemas", "lift"])
top_pairs.sort_values("lift", ascending=False).head(10).round(2)
```



Out[35]:

	etiqueta A	etiqueta B	problemas	lift
113	graphs	shortest paths	164	6.51
130	dfs and similar	trees	309	5.24
133	dfs and similar	dsu	134	5.13
107	graphs	dfs and similar	394	4.78
112	graphs	dsu	149	4.77
138	trees	dsu	88	3.94
128	binary search	two pointers	162	3.83
134	dfs and similar	shortest paths	71	3.37
108	graphs	trees	203	2.88
29	math	number theory	404	2.48

Número de etiquetas por problema y distribución del rating

In [36]:

```
from matplotlib.ticker import PercentFormatter

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

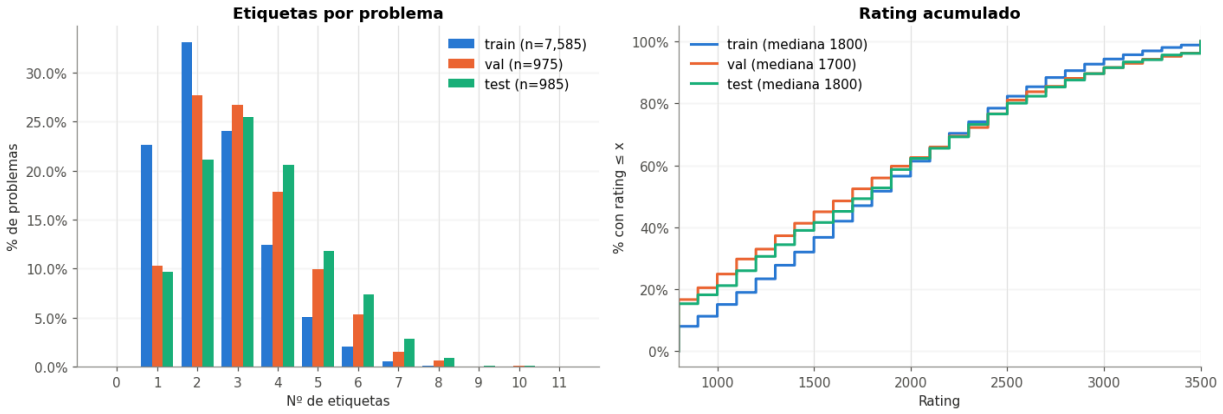
max_tags = max(int(d["tags_kept"].map(len).max()) for d in data.values())
counts = np.arange(0, max_tags + 1)
width = 0.8 / len(SPLITS)
for i, s in enumerate(SPLITS):
    n_tags = data[s]["tags_kept"].map(len)
    props = n_tags.value_counts(normalize=True).reindex(counts, fill_value=0)
    offset = (i - (len(SPLITS) - 1) / 2) * width
    axes[0].bar(counts + offset, props.values, width=width, color=COLORS[s],
                label=f"{s} (n={len(n_tags):,})")
axes[0].set_xticks(counts)
axes[0].set(title="Etiquetas por problema", xlabel="Nº de etiquetas", ylabel="% de problemas")

for s in SPLITS:
    r = np.sort(data[s]["rating"].dropna().to_numpy())
```

```
    axes[1].step(r, np.arange(1, len(r) + 1) / len(r), where="post",
                 linewidth=2, color=COLORS[s], label=f"{s} (mediana {np.median(r):.0f})")
axes[1].set(title="Rating acumulado", xlabel="Rating", ylabel="% con rating ≤ x",
            xlim=(800, 3500))

for ax in axes:
    ax.yaxis.set_major_formatter(PercentFormatter(1))
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="y", alpha=0.3)
    ax.legend(frameon=False)

plt.tight_layout()
plt.show()
```



Longitud en tokens

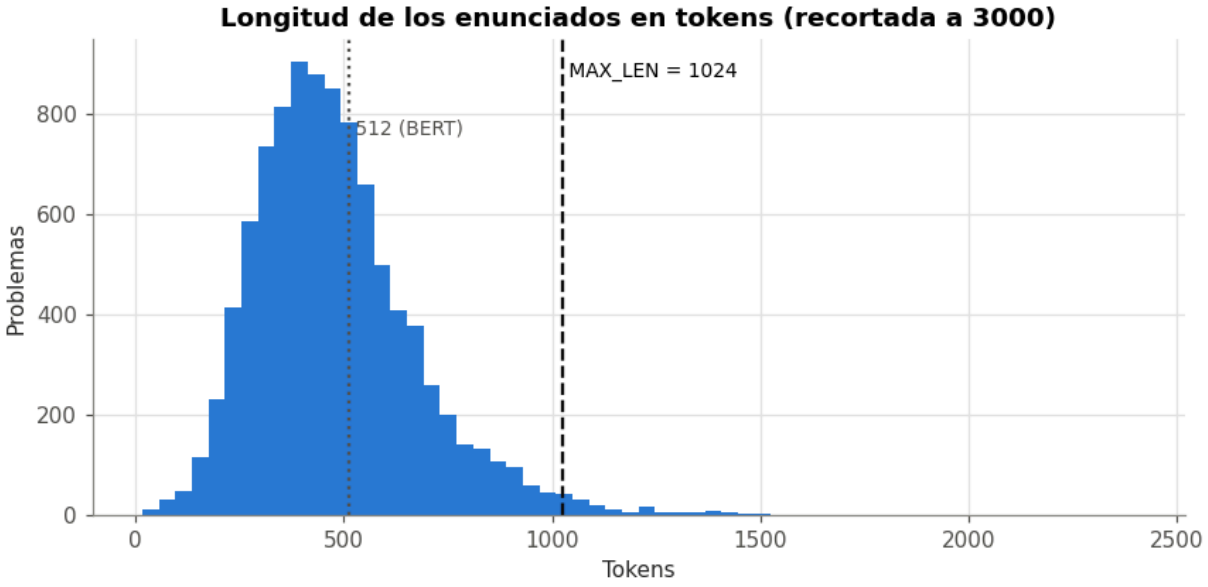
Tokenizamos los textos completos con el tokenizador de ModernBERT para decidir `MAX_LEN` con datos y medir cuánto texto se pierde al truncar.

In [37]:
```
tokenizer = AutoTokenizer.from_pretrained(CONFIG["BASE_MODEL"])
MAX_LEN = CONFIG["MAX_LEN"]
for s in SPLITS:
    data[s]["n_tokens"] = [len(ids) for ids in tokenizer(list(data[s]["text"]))["input_ids"]]

all_lengths = pd.concat([data[s]["n_tokens"] for s in SPLITS])
pct = all_lengths.quantile([0.5, 0.9, 0.95, 0.99]).astype(int)
print("Percentiles de longitud (tokens):", pct.to_dict())
for limit in [512, 1024]:
    print(f"Textos con más de {limit} tokens: {100 * (all_lengths > limit).mean():.1f}%")

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(all_lengths.clip(upper=3000), bins=60, color=COLORS["train"])
ax.axvline(512, color="#52514e", linestyle=":", linewidth=1.5)
ax.axvline(MAX_LEN, color="#0b0b0b", linestyle="--", linewidth=1.5)
ax.text(MAX_LEN, ax.get_ylim()[1] * 0.92, f" MAX_LEN = {MAX_LEN}", fontsize=9)
ax.text(512, ax.get_ylim()[1] * 0.80, " 512 (BERT)", fontsize=9, color="#52514e")
ax.set(title="Longitud de los enunciados en tokens (recortada a 3000)",
       xlabel="Tokens", ylabel="Problemas")
plt.tight_layout()
plt.show()
```

```
Percentiles de longitud (tokens): {0.5: 454, 0.9: 730, 0.95: 850, 0.99: 1090}
Textos con más de 512 tokens: 37.2%
Textos con más de 1024 tokens: 1.6%
```



3.7. Tokenización y datasets para el `Trainer`

Tokenizamos con `truncation=True` y `max_length=MAX_LEN`, **sin padding**: el relleno se añade en cada lote, solo hasta la longitud del texto más largo del lote. Guardamos también la longitud (`length`) para agrupar textos de longitud parecida en el mismo lote.

In [38]:
```
def to_hf_dataset(frame):
    """Tokeniza un split y lo convierte en un `Dataset` con las etiquetas de ambas tareas."""
    enc = tokenizer(list(frame["text"]), truncation=True, max_length=MAX_LEN)
    return Dataset.from_dict({
        "input_ids": enc["input_ids"],
        "attention_mask": enc["attention_mask"],
        "labels_tags": np.stack(frame["y_tags"]).tolist(),
        "labels_rating": frame["rating_z"].tolist(),
        "rating_mask": frame["rating_mask"].tolist(),
        "length": [len(ids) for ids in enc["input_ids"]],
    })


hf = {s: to_hf_dataset(data[s]) for s in SPLITS}
print(hf["train"])
```

```
Dataset({
    features: ['input_ids', 'attention_mask', 'labels_tags', 'labels_rating', 'rating_mask',
'length'],
    num_rows: 7585
})
```

Subimos al Hub el dataset ya procesado (texto limpio, etiquetas conservadas, rating y split), para poder reutilizarlo sin repetir el preprocesamiento.

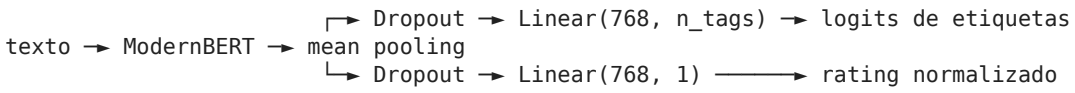
In [39]:
```
if PUSH_TO_HUB:
    keep = ["id", "contest_id", "contest_name", "date", "title", "text", "tags_kept", "rating",
            "split"]
    processed = datasets.DatasetDict({
        s: Dataset.from_pandas(data[s][keep].astype({"date": str}), preserve_index=False)
        for s in SPLITS})
    processed.push_to_hub(CONFIG["HUB_DATASET_ID"])
    print("Dataset procesado subido a", CONFIG["HUB_DATASET_ID"])
```

```
Dataset procesado subido a AlexSA45/codeforces-tags-rating-processed
```

4. Implementación

4.1. Arquitectura del modelo

No existe una clase estándar que resuelva clasificación multi-etiqueta y regresión a la vez, así que definimos una propia:



- **Mean pooling** sobre `attention_mask`: promedia los vectores de todos los tokens reales del texto.

4.2. Función de pérdida

La pérdida combina una **entropía cruzada binaria ponderada** para las etiquetas y un **error cuadrático medio enmascarado** para el rating:

$$\mathcal{L} = \mathrm{BCE}_w(z, y) + \lambda \cdot \frac{\sum_i m_i (\hat{r}_i - r_i)^2}{\max(\sum_i m_i, 1)}$$

- z son los *logits* de las etiquetas e y el vector *multi-hot* real.
- $w_k = \min(\mathrm{neg}_k / \mathrm{pos}_k,\ 10)$ es el peso de los positivos de la etiqueta k, calculado en train. Compensa el desbalance: sin él, el modelo aprendería a no predecir casi nunca las etiquetas raras. El recorte a 10 evita pesos extremos.
- $\hat{r}_i$ y r_i son el rating predicho y el real, ambos normalizados (*z-score*).
- m_i es `rating_mask`: los problemas sin rating no cuentan en la regresión.
- λ (`LAMBDA_RATING` = 0,5) equilibra las dos tareas.

In [40]:
```python
class CFMultiTaskConfig(PretrainedConfig):
    """Configuración del modelo multitarea."""

    model_type = "cf_multitask"

    def __init__(self, base_model_name="answerdotai/ModernBERT-base", encoder_config=None,
                 tag_names=None, lambda_rating=0.5, pos_weight=None, rating_mean=0.0,
                 rating_std=1.0, dropout=0.1, thresholds=None, **kwargs):
        super().__init__(**kwargs)
        self.base_model_name = base_model_name
        self.encoder_config = encoder_config or {}
        self.tag_names = list(tag_names or [])
        self.n_tags = len(self.tag_names)
        self.lambda_rating = lambda_rating
        self.pos_weight = pos_weight
        self.rating_mean = rating_mean
        self.rating_std = rating_std
        self.dropout = dropout
        self.thresholds = thresholds


@dataclass
class CFMultiTaskOutput(ModelOutput):
    loss: Optional[torch.FloatTensor] = None
    tag_logits: Optional[torch.FloatTensor] = None
    rating_pred: Optional[torch.FloatTensor] = None


class CFMultiTaskModel(PreTrainedModel):
    """ModernBERT + mean pooling + dos cabezas (etiquetas y rating)."""

    config_class = CFMultiTaskConfig
    base_model_prefix = "cf_multitask"
    _supports_sdpa = True

    def __init__(self, config):
        super().__init__(config)
        enc_kwargs = dict(config.encoder_config)
        enc_config = AutoConfig.for_model(enc_kwargs.pop("model_type"), **enc_kwargs)

        self.encoder = AutoModel.from_config(enc_config, attn_implementation="sdpa")
        hidden = enc_config.hidden_size
        self.dropout = nn.Dropout(config.dropout)
        self.tags_head = nn.Linear(hidden, config.n_tags)
        self.rating_head = nn.Linear(hidden, 1)
```

```
        pos_weight = config.pos_weight or [1.0] * config.n_tags
        self.register_buffer("pos_weight", torch.tensor(pos_weight, dtype=torch.float32))
        self.post_init()

    def _init_weights(self, module):
        if module in (self.tags_head, self.rating_head):
            module.weight.data.normal_(mean=0.0, std=0.02)
            module.bias.data.zero_()

    @classmethod
    def from_base(cls, base_model_name, **config_kwargs):
        """Crea el modelo para entrenar: encoder preentrenado + cabezas nuevas."""
        enc_config = AutoConfig.from_pretrained(base_model_name)
        config = CFMultiTaskConfig(base_model_name=base_model_name,
                                   encoder_config=enc_config.to_dict(), **config_kwargs)
        model = cls(config)
        model.encoder = AutoModel.from_pretrained(base_model_name, attn_implementation="sdpa",
                                                  dtype=torch.float32)
        return model

    def forward(self, input_ids=None, attention_mask=None, labels_tags=None,
                labels_rating=None, rating_mask=None, **kwargs):
        hidden = self.encoder(input_ids=input_ids,
attention_mask=attention_mask).last_hidden_state

        mask = attention_mask.unsqueeze(-1).to(hidden.dtype)
        pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1e-6)
        pooled = self.dropout(pooled)
        tag_logits = self.tags_head(pooled)
        rating_pred = self.rating_head(pooled).squeeze(-1)

        loss = None
        if labels_tags is not None:

            bce = F.binary_cross_entropy_with_logits(
                tag_logits.float(), labels_tags.float(), pos_weight=self.pos_weight)
            m = rating_mask.float()
            mse = (m * (rating_pred.float() - labels_rating.float()) ** 2).sum() / m.sum().clamp(
                min=1.0)
            loss = bce + self.config.lambda_rating * mse
        return CFMultiTaskOutput(loss=loss, tag_logits=tag_logits, rating_pred=rating_pred)


Y_train = np.stack(data["train"]["y_tags"])
pos = Y_train.sum(axis=0)
POS_WEIGHT = np.minimum((len(Y_train) - pos) / np.maximum(pos, 1),
                        CONFIG["POS_WEIGHT_CLIP"]).round(3).tolist()
pd.DataFrame({"positivos en train": pos.astype(int), "pos_weight": POS_WEIGHT}, index=TAGS).T
```

Out[40]:

	implementation	math	greedy	dp	data structures	constructive algorithms	brute force	graphs	sortings	binary search	dfs and similar	trees	strings	number theory	combinatorics	bitmasks	two pointers	geometry	dsu	shortest paths
positivos en train	2252.000	2194.000	2137.000	1639.000	1356.000	1308.000	1252.000	865.000	826.000	796.000	723.000	619.0	579.0	563.0	498.0	412.0	403.0	329.0	274.0	221.0
pos_weight	2.368	2.457	2.549	3.628	4.594	4.799	5.058	7.769	8.183	8.529	9.491	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0

4.3. *Collator* y métricas

El *collator* forma cada lote: rellena `input_ids` y `attention_mask` hasta la longitud máxima del lote y apila las tres etiquetas en tensores `float32`.

Las métricas se definen una sola vez y se usan para todos los modelos, para que la comparación sea justa:

- **Etiquetas:** F1 micro (dominado por las etiquetas frecuentes), F1 macro (media por etiqueta, sensible a las raras), F1 por muestra, mAP macro (calidad del orden de las probabilidades, independiente del umbral) y *subset accuracy* (acertar exactamente el conjunto de etiquetas).
- **Rating** (solo con `rating_mask = 1`): MAE, RMSE, correlación de Spearman y % de predicciones con error absoluto ≤ 200.

In [41]:

```
class MultiTaskCollator:
    """Padding dinámico de los textos + tensores float32 con las etiquetas."""

    def __init__(self, tokenizer):
        self.tokenizer = tokenizer
```

```
    def __call__(self, features):
        batch = self.tokenizer.pad(
            [{"input_ids": f["input_ids"], "attention_mask": f["attention_mask"]}
             for f in features],
            padding=True, pad_to_multiple_of=8, return_tensors="pt")
        for key in ["labels_tags", "labels_rating", "rating_mask"]:
            batch[key] = torch.tensor([f[key] for f in features], dtype=torch.float32)
        return batch


collator = MultiTaskCollator(tokenizer)


def tag_metrics(y_true, scores, thresholds=0.5):
    """Métricas multi-etiqueta; `thresholds` puede ser un escalar o un vector por etiqueta."""
    y_pred = (scores >= np.asarray(thresholds)).astype(int)
    has_pos = y_true.sum(axis=0) > 0
    return {
        "f1_micro": f1_score(y_true, y_pred, average="micro", zero_division=0),
        "f1_macro": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "f1_samples": f1_score(y_true, y_pred, average="samples", zero_division=0),
        "map_macro": average_precision_score(y_true[:, has_pos], scores[:, has_pos],
                                             average="macro"),
        "subset_acc": float((y_pred == y_true).all(axis=1).mean()),
    }


def rating_metrics(r_true, r_pred, mask):
    """Métricas de regresión en la escala original, solo donde hay rating."""
    m = mask.astype(bool)
    err = r_pred[m] - r_true[m]
    return {
        "mae": float(np.abs(err).mean()),
        "rmse": float(np.sqrt((err ** 2).mean())),
        "spearman": float(spearmanr(r_true[m], r_pred[m]).correlation),
        "within_200": float((np.abs(err) <= 200).mean()),
    }


def compute_metrics(eval_pred):
    """Métricas de validación durante el entrenamiento (umbral fijo 0.5)."""
    tag_logits, rating_pred = eval_pred.predictions
    y_tags, y_rating, mask = eval_pred.label_ids
    scores = 1 / (1 + np.exp(-tag_logits))
    out = tag_metrics(y_tags, scores, 0.5)
    out.update(rating_metrics(y_rating * RATING_STD + RATING_MEAN,
                              rating_pred * RATING_STD + RATING_MEAN, mask))
    return out
```

4.4. Hiperparámetros

Parámetro	Valor	Justificación
`max_length`	1024	Cubre la mayoría de enunciados; el resto se trunca por el final
`learning_rate`	5e-5	Valor habitual para el ajuste fino completo de *encoders* de este tamaño
`per_device_train_batch_size`	8	Cabe en la T4 con 1024 tokens en fp16
`gradient_accumulation_steps`	2	Lote efectivo de 16
`num_train_epochs`	5	Con *early stopping* (paciencia 2)
`warmup_steps`	0.1	10 % de pasos de calentamiento y luego decaimiento lineal
`weight_decay`	0.01	Regularización estándar de AdamW
`lambda_rating`	0.5	Peso de la pérdida de rating
`fp16`	True	Precisión mixta; la T4 no soporta bf16
`train_sampling_strategy`	`group_by_length`	Agrupa textos de longitud parecida: menos padding
`metric_for_best_model`	`f1_macro` (val, umbral 0.5)	Se queda con la mejor época
`hub_strategy`	`every_save`	Sube el modelo al Hub en cada época

In [42]:
```
training_args = TrainingArguments(
    output_dir="runs/main",
    num_train_epochs=CONFIG["EPOCHS"],
    max_steps=MAX_STEPS,
    learning_rate=CONFIG["LR"],
    per_device_train_batch_size=CONFIG["BATCH_SIZE"],
    per_device_eval_batch_size=2 * CONFIG["BATCH_SIZE"],
    gradient_accumulation_steps=CONFIG["GRAD_ACCUM"],
    warmup_steps=CONFIG["WARMUP_RATIO"],
    lr_scheduler_type="linear",
```

```
    weight_decay=CONFIG["WEIGHT_DECAY"],
    fp16=torch.cuda.is_available(),
    train_sampling_strategy="group_by_length",
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
    greater_is_better=True,
    label_names=["labels_tags", "labels_rating", "rating_mask"],
    remove_unused_columns=False,
    logging_steps=50,
    report_to="none",
    seed=SEED,
    push_to_hub=PUSH_TO_HUB,
    hub_model_id=CONFIG["HUB_MODEL_ID"] if PUSH_TO_HUB else None,
    hub_strategy="every_save",
)
```

4.5. Ajuste fino

Con `RUN_TRAINING = True` se crea el modelo a partir de ModernBERT-base y se entrena; con `False` se descarga el ya ajustado desde el Hub junto con el historial del entrenamiento. Al terminar, el `Trainer` recarga los pesos de la **mejor época** según el F1 macro de validación.

In [43]:
```
if CONFIG["RUN_TRAINING"]:
    model = CFMultiTaskModel.from_base(
        CONFIG["BASE_MODEL"], tag_names=TAGS, lambda_rating=CONFIG["LAMBDA_RATING"],
        pos_weight=POS_WEIGHT, rating_mean=RATING_MEAN, rating_std=RATING_STD,
        dropout=CONFIG["DROPOUT"])
    trainer = Trainer(
        model=model, args=training_args, train_dataset=hf["train"], eval_dataset=hf["val"],
        data_collator=collator, processing_class=tokenizer, compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(CONFIG["EARLY_STOPPING_PATIENCE"])],
    )
    t0 = time.time()
    trainer.train()
    train_seconds = time.time() - t0
    model = trainer.model
    log_history = trainer.state.log_history
    train_info = {"seconds": train_seconds,
                  "gpu": torch.cuda.get_device_name(0) if torch.cuda.is_available() else "cpu"}
    print(f"Entrenamiento completado en {train_seconds / 60:.1f} min en {train_info['gpu']}")
else:
    model = CFMultiTaskModel.from_pretrained(CONFIG["HUB_MODEL_ID"], dtype=torch.float32)
    log_path = hf_hub_download(CONFIG["HUB_MODEL_ID"], "training_log.json")
    with open(log_path) as fh:
        saved = json.load(fh)
    log_history, train_info = saved["log_history"], saved["train_info"]
    print(f"Modelo descargado de {CONFIG['HUB_MODEL_ID']} "
          f"(entrenado en {train_info['seconds'] / 60:.1f} min en {train_info['gpu']})")

model = model.to(DEVICE).eval()
print(f"Parámetros del modelo: {sum(p.numel() for p in model.parameters()) / 1e6:.1f} M "
      "(todos entrenables)")
```

```
{'loss': '1.674', 'grad_norm': '10.29', 'learning_rate': '1.029e-05', 'epoch': '0.1054'}
{'loss': '1.582', 'grad_norm': '16.36', 'learning_rate': '2.08e-05', 'epoch': '0.2107'}
{'loss': '1.526', 'grad_norm': '13.63', 'learning_rate': '3.13e-05', 'epoch': '0.3161'}
{'loss': '1.453', 'grad_norm': '4.819', 'learning_rate': '4.181e-05', 'epoch': '0.4215'}
{'loss': '1.384', 'grad_norm': '6.749', 'learning_rate': '4.974e-05', 'epoch': '0.5269'}
{'loss': '1.318', 'grad_norm': '15', 'learning_rate': '4.857e-05', 'epoch': '0.6322'}
{'loss': '1.365', 'grad_norm': '11.77', 'learning_rate': '4.74e-05', 'epoch': '0.7376'}
{'loss': '1.266', 'grad_norm': '11.75', 'learning_rate': '4.623e-05', 'epoch': '0.843'}
{'loss': '1.208', 'grad_norm': '7.591', 'learning_rate': '4.506e-05', 'epoch': '0.9484'}
{'eval_loss': '1.443', 'eval_f1_micro': '0.4636', 'eval_f1_macro': '0.43', 'eval_f1_samples':
'0.4547', 'eval_map_macro': '0.4101', 'eval_subset_acc': '0', 'eval_mae': '510.7', 'eval_rmse':
'680.9', 'eval_spearman': '0.6798', 'eval_within_200': '0.2709', 'eval_runtime': '19.7',
'eval_samples_per_second': '49.5', 'eval_steps_per_second': '3.097', 'epoch': '1'}
```

{'loss': '1.243', 'grad_norm': '3.438', 'learning_rate': '4.389e-05', 'epoch': '1.053'}
{'loss': '1.065', 'grad_norm': '3.44', 'learning_rate': '4.272e-05', 'epoch': '1.158'}
{'loss': '1.075', 'grad_norm': '4.966', 'learning_rate': '4.155e-05', 'epoch': '1.263'}
{'loss': '1.101', 'grad_norm': '5.462', 'learning_rate': '4.038e-05', 'epoch': '1.369'}
{'loss': '1.126', 'grad_norm': '4.394', 'learning_rate': '3.921e-05', 'epoch': '1.474'}
{'loss': '1.103', 'grad_norm': '7.2', 'learning_rate': '3.804e-05', 'epoch': '1.58'}
{'loss': '1.096', 'grad_norm': '6.949', 'learning_rate': '3.687e-05', 'epoch': '1.685'}
{'loss': '1.101', 'grad_norm': '5.647', 'learning_rate': '3.57e-05', 'epoch': '1.79'}
{'loss': '1.1', 'grad_norm': '6.111', 'learning_rate': '3.453e-05', 'epoch': '1.896'}
{'loss': '1.037', 'grad_norm': '9.912', 'learning_rate': '3.336e-05', 'epoch': '2'}
{'eval_loss': '1.19', 'eval_f1_micro': '0.4992', 'eval_f1_macro': '0.4697', 'eval_f1_samples':
'0.4883', 'eval_map_macro': '0.4874', 'eval_subset_acc': '0.003077', 'eval_mae': '416.1',
'eval_rmse': '545.2', 'eval_spearman': '0.7656', 'eval_within_200': '0.3316', 'eval_runtime':
'19.8', 'eval_samples_per_second': '49.25', 'eval_steps_per_second': '3.081', 'epoch': '2'}
{'loss': '0.9592', 'grad_norm': '8.371', 'learning_rate': '3.219e-05', 'epoch': '2.105'}
{'loss': '0.9057', 'grad_norm': '7.408', 'learning_rate': '3.102e-05', 'epoch': '2.211'}
{'loss': '0.9453', 'grad_norm': '17.29', 'learning_rate': '2.985e-05', 'epoch': '2.316'}
{'loss': '0.9147', 'grad_norm': '6.382', 'learning_rate': '2.869e-05', 'epoch': '2.421'}
{'loss': '0.936', 'grad_norm': '7.125', 'learning_rate': '2.752e-05', 'epoch': '2.527'}
{'loss': '0.9091', 'grad_norm': '3.059', 'learning_rate': '2.635e-05', 'epoch': '2.632'}
{'loss': '0.9016', 'grad_norm': '6.162', 'learning_rate': '2.518e-05', 'epoch': '2.738'}
{'loss': '0.8743', 'grad_norm': '3.34', 'learning_rate': '2.401e-05', 'epoch': '2.843'}
{'loss': '0.8784', 'grad_norm': '8.842', 'learning_rate': '2.284e-05', 'epoch': '2.948'}
{'eval_loss': '1.179', 'eval_f1_micro': '0.5094', 'eval_f1_macro': '0.4938', 'eval_f1_samples':
'0.5009', 'eval_map_macro': '0.5183', 'eval_subset_acc': '0.003077', 'eval_mae': '428.9',
'eval_rmse': '573.6', 'eval_spearman': '0.7817', 'eval_within_200': '0.3429', 'eval_runtime':
'19.78', 'eval_samples_per_second': '49.3', 'eval_steps_per_second': '3.084', 'epoch': '3'}
{'loss': '0.8773', 'grad_norm': '5.106', 'learning_rate': '2.167e-05', 'epoch': '3.053'}
{'loss': '0.7472', 'grad_norm': '4.746', 'learning_rate': '2.05e-05', 'epoch': '3.158'}
{'loss': '0.7478', 'grad_norm': '5.469', 'learning_rate': '1.933e-05', 'epoch': '3.263'}
{'loss': '0.7333', 'grad_norm': '4.566', 'learning_rate': '1.816e-05', 'epoch': '3.369'}
{'loss': '0.7109', 'grad_norm': '3.316', 'learning_rate': '1.699e-05', 'epoch': '3.474'}
{'loss': '0.7075', 'grad_norm': '3.929', 'learning_rate': '1.582e-05', 'epoch': '3.58'}
{'loss': '0.7133', 'grad_norm': '6.022', 'learning_rate': '1.465e-05', 'epoch': '3.685'}
{'loss': '0.6956', 'grad_norm': '5.937', 'learning_rate': '1.348e-05', 'epoch': '3.79'}
{'loss': '0.7149', 'grad_norm': '3.925', 'learning_rate': '1.231e-05', 'epoch': '3.896'}
{'loss': '0.6562', 'grad_norm': '2.771', 'learning_rate': '1.114e-05', 'epoch': '4'}
{'eval_loss': '1.151', 'eval_f1_micro': '0.5247', 'eval_f1_macro': '0.5046', 'eval_f1_samples':
'0.515', 'eval_map_macro': '0.5172', 'eval_subset_acc': '0.01128', 'eval_mae': '426.5',
'eval_rmse': '533.9', 'eval_spearman': '0.7858', 'eval_within_200': '0.2791', 'eval_runtime':
'19.84', 'eval_samples_per_second': '49.14', 'eval_steps_per_second': '3.075', 'epoch': '4'}
{'loss': '0.5881', 'grad_norm': '6.672', 'learning_rate': '9.967e-06', 'epoch': '4.105'}
{'loss': '0.5488', 'grad_norm': '2.911', 'learning_rate': '8.797e-06', 'epoch': '4.211'}
{'loss': '0.5607', 'grad_norm': '4.956', 'learning_rate': '7.628e-06', 'epoch': '4.316'}
{'loss': '0.5771', 'grad_norm': '3.791', 'learning_rate': '6.458e-06', 'epoch': '4.421'}
{'loss': '0.5764', 'grad_norm': '4.595', 'learning_rate': '5.288e-06', 'epoch': '4.527'}
{'loss': '0.5637', 'grad_norm': '3.842', 'learning_rate': '4.118e-06', 'epoch': '4.632'}
{'loss': '0.5477', 'grad_norm': '3.554', 'learning_rate': '2.948e-06', 'epoch': '4.738'}
{'loss': '0.5557', 'grad_norm': '4.47', 'learning_rate': '1.778e-06', 'epoch': '4.843'}
{'loss': '0.5598', 'grad_norm': '6.001', 'learning_rate': '6.083e-07', 'epoch': '4.948'}
{'eval_loss': '1.183', 'eval_f1_micro': '0.5369', 'eval_f1_macro': '0.5141', 'eval_f1_samples':
'0.528', 'eval_map_macro': '0.5147', 'eval_subset_acc': '0.01333', 'eval_mae': '407.8',
'eval_rmse': '539.2', 'eval_spearman': '0.7802', 'eval_within_200': '0.3378', 'eval_runtime':
'19.72', 'eval_samples_per_second': '49.44', 'eval_steps_per_second': '3.093', 'epoch': '5'}
{'train_runtime': '2116', 'train_samples_per_second': '17.92', 'train_steps_per_second': '1.122',
'train_loss': '0.9411', 'epoch': '5'}
Entrenamiento completado en 35.5 min en Tesla T4
Parámetros del modelo: 149.0 M (todos entrenables)

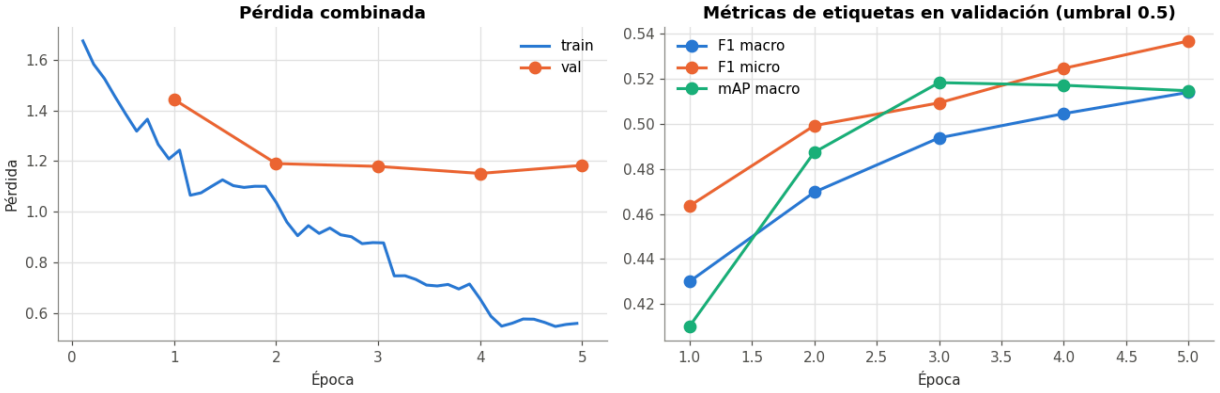
Curvas de entrenamiento

In [44]:
```
logs = pd.DataFrame(log_history)
train_logs = logs.dropna(subset=["loss"]) if "loss" in logs else pd.DataFrame()
eval_logs = logs.dropna(subset=["eval_loss"]).set_index("epoch")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
if len(train_logs):
    axes[0].plot(train_logs["epoch"], train_logs["loss"], color=COLORS["train"], linewidth=2,
                 label="train")
axes[0].plot(eval_logs.index, eval_logs["eval_loss"], color=COLORS["val"], linewidth=2,
             marker="o", markersize=8, label="val")
axes[0].set(title="Pérdida combinada", xlabel="Época", ylabel="Pérdida")
axes[0].legend(frameon=False)
metric_style = {"eval_f1_macro": ("F1 macro", "#2a78d6"), "eval_f1_micro": ("F1 micro",
"#eb6834"),
                "eval_map_macro": ("mAP macro", "#1baf7a")}
```

```
for col, (label, color) in metric_style.items():
    axes[1].plot(eval_logs.index, eval_logs[col], color=color, linewidth=2, marker="o",
                 markersize=8, label=label)
axes[1].set(title="Métricas de etiquetas en validación (umbral 0.5)", xlabel="Época")
axes[1].legend(frameon=False)
plt.tight_layout()
plt.show()

eval_cols = ["eval_loss", "eval_f1_micro", "eval_f1_macro", "eval_map_macro", "eval_mae",
             "eval_spearman"]
eval_logs[eval_cols].round(4)
```



Out[44]:

epoch	eval_loss	eval_f1_micro	eval_f1_macro	eval_map_macro	eval_mae	eval_spearman
1.0	1.4428	0.4636	0.4300	0.4101	510.6656	0.6798
2.0	1.1902	0.4992	0.4697	0.4874	416.0790	0.7656
3.0	1.1790	0.5094	0.4938	0.5183	428.9177	0.7817
4.0	1.1513	0.5247	0.5046	0.5172	426.4944	0.7858
5.0	1.1831	0.5369	0.5141	0.5147	407.8499	0.7802

Análisis del entrenamiento.

- El ajuste fino tardó **35,5 minutos** en una T4 y completó las 5 épocas: el *early stopping* no llegó a activarse porque el F1 macro de validación mejoró en todas ellas.
- La pérdida de train baja de forma continua, de 1,674 en los primeros pasos a 0,560 al final. La de validación, en cambio, se estanca pronto: baja de 1,443 a 1,190 entre las épocas 1 y 2, alcanza su mínimo en la época 4 (1,151) y vuelve a subir en la 5 (1,183). Es una señal de **inicio de sobreajuste**: el modelo se sigue ajustando a train, pero ya no generaliza mejor.
- El **mAP** de validación, que mide la calidad del orden de las probabilidades y no depende del umbral, se estabiliza desde la época 3 (0,518 → 0,517 → 0,515). El F1 con umbral 0,5 sigue subiendo (macro 0,494 → 0,505 → 0,514) porque las probabilidades se vuelven más extremas y cruzan el umbral con más frecuencia; los umbrales por etiqueta de la sección 5 corrigen parte de ese efecto.
- El `Trainer` se queda con la **época 5**, la de mayor F1 macro en validación (0,514). En el rating, el MAE de validación baja de 511 a 408 y la correlación de Spearman sube de 0,68 a 0,78.

5. Resultados y discusión

5.1. Predicciones y ajuste de umbrales

El modelo devuelve una probabilidad por etiqueta. Usar el mismo umbral (0.5) para todas no es óptimo: las etiquetas raras suelen necesitar umbrales distintos. Ajustamos un **umbral por etiqueta en validación** (rejilla de 0.05 a 0.95 con paso 0.05, maximizando el F1 de cada etiqueta) y lo aplicamos sin cambios a test.

Para la inferencia ordenamos los textos por longitud: los lotes tienen menos relleno y la predicción es más rápida.

In [45]:
```
def sorted_batches(ds, batch_size):
    """Itera por lotes ordenados por longitud; devuelve también el orden para deshacerlo."""
    order = np.argsort(np.asarray(ds["length"]))
    loader = DataLoader(ds.select(order), batch_size=batch_size, collate_fn=collator)
    return order, loader
```

```
@torch.no_grad()
def predict(model, ds, batch_size=32):
    """Probabilidades de cada etiqueta y rating (escala original) en el orden de `ds`."""
    model.eval()
    order, loader = sorted_batches(ds, batch_size)
    probs, ratings = [], []
    for batch in loader:
        with torch.autocast(DEVICE.type, dtype=torch.float16, enabled=DEVICE.type == "cuda"):
            out = model(input_ids=batch["input_ids"].to(DEVICE),
                        attention_mask=batch["attention_mask"].to(DEVICE))
        probs.append(torch.sigmoid(out.tag_logits.float()).cpu().numpy())
        ratings.append(out.rating_pred.float().cpu().numpy())
    probs, ratings = np.concatenate(probs), np.concatenate(ratings)
    inverse = np.argsort(order)
    return probs[inverse], ratings[inverse] * RATING_STD + RATING_MEAN


THRESHOLD_GRID = np.round(np.arange(0.05, 0.951, 0.05), 2)


def tune_thresholds(y_true, scores):
    """Umbral por etiqueta que maximiza su F1 en validación."""
    thresholds = []
    for k in range(y_true.shape[1]):
        f1s = [f1_score(y_true[:, k], scores[:, k] >= t, zero_division=0)
               for t in THRESHOLD_GRID]
        thresholds.append(float(THRESHOLD_GRID[int(np.argmax(f1s))]))
    return np.array(thresholds)


Y = {s: np.stack(data[s]["y_tags"]) for s in SPLITS}
R = {s: data[s]["rating"].fillna(0).to_numpy(dtype=float) for s in SPLITS}
M = {s: data[s]["rating_mask"].to_numpy() for s in SPLITS}

ft_val_probs, ft_val_rating = predict(model, hf["val"])
ft_test_probs, ft_test_rating = predict(model, hf["test"])
ft_thresholds = tune_thresholds(Y["val"], ft_val_probs)
pd.Series(ft_thresholds, index=TAGS, name="umbral").to_frame().T
```

Out[45]:

	implementation	math	greedy	dp	data structures	constructive algorithms	brute force	graphs	sortings	binary search	dfs and similar	trees	strings	number theory	combinatorics	bitmasks	two pointers	geometry	dsu	shortest paths
umbral	0.3	0.6	0.6	0.6	0.65	0.7	0.4	0.6	0.7	0.7	0.6	0.7	0.75	0.8	0.7	0.7	0.5	0.75	0.55	0.65

Guardamos los umbrales en la configuración del modelo y subimos la versión final al Hub, junto con el historial del entrenamiento.

In [46]:
```
model.config.thresholds = ft_thresholds.tolist()
if PUSH_TO_HUB and CONFIG["RUN_TRAINING"]:
    model.push_to_hub(CONFIG["HUB_MODEL_ID"], commit_message="Modelo final con umbrales")
    tokenizer.push_to_hub(CONFIG["HUB_MODEL_ID"])
    log_file = "training_log.json"
    with open(log_file, "w") as fh:
        json.dump({"log_history": log_history, "train_info": train_info}, fh)
    HfApi().upload_file(path_or_fileobj=log_file, path_in_repo=log_file,
                        repo_id=CONFIG["HUB_MODEL_ID"])
    print("Modelo, tokenizador, umbrales e historial subidos a", CONFIG["HUB_MODEL_ID"])
```

```
Modelo, tokenizador, umbrales e historial subidos a AlexSA45/modernbert-codeforces-tags-rating
```

Uso del modelo publicado

Esta celda carga el modelo **desde el Hub** y predice las etiquetas y la dificultad de un enunciado de test. Es todo lo que necesita un usuario del modelo: el tokenizador, la clase `CFMultiTaskModel` y los umbrales guardados en la configuración.

In [47]:
```
def predict_problem(model, text):
    """Etiquetas (con su probabilidad) y rating predichos para un enunciado ya limpio."""
    enc = tokenizer(text, truncation=True, max_length=MAX_LEN, return_tensors="pt").to(DEVICE)
    with torch.no_grad():
        out = model(**enc)
    probs = torch.sigmoid(out.tag_logits.float())[0].cpu().numpy()
    thr = np.array(model.config.thresholds or [0.5] * model.config.n_tags)
    tags = {model.config.tag_names[k]: round(float(probs[k]), 3)
            for k in np.argsort(-probs) if probs[k] >= thr[k]}
```

```
    rating = float(out.rating_pred[0]) * model.config.rating_std + model.config.rating_mean
    return tags, rating


if PUSH_TO_HUB or not CONFIG["RUN_TRAINING"]:
    demo_model = CFMultiTaskModel.from_pretrained(CONFIG["HUB_MODEL_ID"]).to(DEVICE).eval()
else:
    demo_model = model
sample = data["test"].iloc[0]
pred_tags, pred_rating = predict_problem(demo_model, sample["text"])
print(sample["text"][:500], "...\n")
print("Etiquetas reales :", sample["tags_kept"], "| rating real:", sample["rating"])
print("Etiquetas predichas:", pred_tags, f"| rating predicho: {pred_rating:.0f}")
if demo_model is not model:
    del demo_model
```

```
Title: Username
Input: The first line contains a single integer t (1 <= t <= 10^3) — the number of test cases. The
only line of each test case contains a string s (2 <= |s| <= 50), consisting of lowercase Latin
letters and/or digits. Additional constraint on input: there is at least one valid username that
can generate the given ID.
Output: For each test case on a separate line, print a string — any valid username that can
generate the given ID. If there are multiple correct usernames, you can p ...

Etiquetas reales : ['implementation'] | rating real: 1100.0
Etiquetas predichas: {'implementation': 0.825, 'strings': 0.809, 'constructive algorithms': 0.715,
'greedy': 0.631, 'brute force': 0.554} | rating predicho: 804
```

5.2. Baselines

Para saber si el ajuste fino aporta algo hay que compararlo con referencias razonables. Todas se evalúan sobre el mismo test, con las mismas métricas y con umbrales por etiqueta ajustados en validación:

- **B0, trivial.** Cada etiqueta recibe como puntuación su frecuencia en train y el rating es siempre la media de train. Es el mínimo que cualquier modelo debe superar.
- **B1, TF-IDF + modelos lineales.** Bolsa de unigramas y bigramas con regresión logística y regresión `Ridge` para el rating. Es el baseline clásico en clasificación de textos.
- **B2, ModernBERT sin ajustar (*linear probe*).** Es el modelo base sin ajustar: se extraen los *embeddings* de ModernBERT-base congelado y se entrenan encima los mismos modelos lineales. Mide cuánto aporta ajustar el *encoder* frente a usar solo sus representaciones preentrenadas.

In [48]:
```
results = {}


def register(name, val_scores, test_scores, val_rating, test_rating):
    """Guarda las salidas de un modelo y ajusta sus umbrales en validación."""
    results[name] = {
        "val_scores": val_scores, "test_scores": test_scores,
        "val_rating": val_rating, "test_rating": test_rating,
        "thresholds": tune_thresholds(Y["val"], val_scores),
    }


register("ModernBERT ajustado", ft_val_probs, ft_test_probs, ft_val_rating, ft_test_rating)

# B0: frecuencia a priori de cada etiqueta y media del rating
prior = Y["train"].mean(axis=0)
register("B0 Trivial",
         np.tile(prior, (len(Y["val"]), 1)), np.tile(prior, (len(Y["test"]), 1)),
         np.full(len(Y["val"]), RATING_MEAN), np.full(len(Y["test"]), RATING_MEAN))


def fit_linear_baselines(X_train, X_val, X_test, name):
    """Regresión logística por etiqueta (C elegido por mAP en val) + Ridge para el rating."""
    best = None
    for C in [0.1, 1, 10]:
        clf = OneVsRestClassifier(LogisticRegression(
            C=C, class_weight="balanced", solver="liblinear", max_iter=2000))
        clf.fit(X_train, Y["train"])
        val_scores = clf.predict_proba(X_val)
        val_map = tag_metrics(Y["val"], val_scores)["map_macro"]
        print(f"  {name}: C={C:<4} mAP macro val = {val_map:.4f}")
        if best is None or val_map > best[0]:
```

```
            best = (val_map, C, clf, val_scores)
    _, C, clf, val_scores = best
    m = M["train"].astype(bool)
    ridge = Ridge(alpha=1.0).fit(X_train[m], R["train"][m])
    register(name, val_scores, clf.predict_proba(X_test), ridge.predict(X_val),
             ridge.predict(X_test))
    print(f"  {name}: C elegido = {C}")


# B1: TF-IDF
tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=50_000, sublinear_tf=True)
X_tfidf = {"train": tfidf.fit_transform(data["train"]["text"])}
for s in ["val", "test"]:
    X_tfidf[s] = tfidf.transform(data[s]["text"])
fit_linear_baselines(X_tfidf["train"], X_tfidf["val"], X_tfidf["test"], "TF-IDF + LR")
```

```
 TF-IDF + LR: C=0.1  mAP macro val = 0.4855
 TF-IDF + LR: C=1    mAP macro val = 0.5066
 TF-IDF + LR: C=10   mAP macro val = 0.4951
 TF-IDF + LR: C elegido = 1
```

Para el *linear probe* extraemos los *embeddings* de ModernBERT-base congelado, en fp16 y sin gradientes, con el mismo *mean pooling* que el modelo ajustado.

In [49]:
```
@torch.no_grad()
def embed(encoder, ds, batch_size=32):
    """Embeddings (mean pooling) de un encoder congelado, en el orden de `ds`."""
    order, loader = sorted_batches(ds, batch_size)
    out = []
    for batch in loader:
        ids, att = batch["input_ids"].to(DEVICE), batch["attention_mask"].to(DEVICE)
        hidden = encoder(input_ids=ids, attention_mask=att).last_hidden_state.float()
        mask = att.unsqueeze(-1).float()
        out.append(((hidden * mask).sum(1) / mask.sum(1)).cpu().numpy())
    return np.concatenate(out)[np.argsort(order)]


frozen = AutoModel.from_pretrained(
    CONFIG["BASE_MODEL"], attn_implementation="sdpa",
    dtype=torch.float16 if DEVICE.type == "cuda" else torch.float32).to(DEVICE).eval()
t0 = time.time()
X_emb = {s: embed(frozen, hf[s]) for s in SPLITS}
print(f"Embeddings extraídos en {time.time() - t0:.0f} s; forma de train:
{X_emb['train'].shape}")
del frozen
torch.cuda.empty_cache()

scaler = StandardScaler().fit(X_emb["train"])
X_emb = {s: scaler.transform(x) for s, x in X_emb.items()}
fit_linear_baselines(X_emb["train"], X_emb["val"], X_emb["test"], "Linear probe")
```

```
Embeddings extraídos en 138 s; forma de train: (7585, 768)
  Linear probe: C=0.1  mAP macro val = 0.3985
  Linear probe: C=1    mAP macro val = 0.3667
  Linear probe: C=10   mAP macro val = 0.3497
  Linear probe: C elegido = 0.1
```

5.3. Tabla principal de resultados

Todos los modelos sobre el **test completo**, con umbrales ajustados en validación.

In [50]:
```
ROW_ORDER = ["B0 Trivial", "TF-IDF + LR", "Linear probe", "ModernBERT ajustado"]


def evaluate(name, split="test", thresholds=None):
    """Métricas de un modelo en un split (por defecto, con sus umbrales ajustados)."""
    res = results[name]
    thr = res["thresholds"] if thresholds is None else thresholds
    row = tag_metrics(Y[split], res[f"{split}_scores"], thr)
    row.update(rating_metrics(R[split], res[f"{split}_rating"], M[split]))
    return row


main_table = pd.DataFrame({name: evaluate(name) for name in ROW_ORDER}).T
main_table.columns = ["F1 micro", "F1 macro", "F1 samples", "mAP macro", "Subset acc", "MAE",
```

```
                      "RMSE", "Spearman", "% |err|≤200"]
print(f"Test: {len(Y['test'])} problemas")
main_table.round(3)
```

```
Test: 985 problemas
```

Out[50]:

	F1 micro	F1 macro	F1 samples	mAP macro	Subset acc	MAE	RMSE	Spearman	% |err|≤200
B0 Trivial	0.326	0.266	0.317	0.172	0.000	670.429	791.320	NaN	0.171
TF-IDF + LR	0.554	0.519	0.537	0.515	0.010	470.739	606.215	0.681	0.304
Linear probe	0.483	0.444	0.472	0.408	0.004	521.497	655.043	0.604	0.246
ModernBERT ajustado	0.545	0.530	0.535	0.528	0.029	442.000	584.129	0.713	0.333

Discusión de la tabla principal.

- El **modelo ajustado** es el mejor en F1 macro (0,530), mAP macro (0,528) y *subset accuracy* (0,029), y en **todas las métricas de rating**: MAE 442, RMSE 584, Spearman 0,713 y un 33,3 % de predicciones con error ≤ 200.
- **Frente al modelo base sin ajustar (*linear probe*)** la mejora es clara en todo: +0,086 de F1 macro, +0,120 de mAP y 79 puntos menos de MAE. Las representaciones preentrenadas de ModernBERT, usadas tal cual, no bastan: lo que aporta el rendimiento es ajustar el *encoder*. De hecho, el *linear probe* queda por debajo de TF-IDF; los *embeddings* genéricos, promediados sobre todo el texto, recogen peor las palabras clave del dominio que una bolsa de palabras.
- **Frente a TF-IDF** la diferencia en etiquetas es pequeña: el modelo ajustado gana en F1 macro (+0,011) y mAP (+0,013), empata en F1 por muestra (0,535 frente a 0,537) y pierde ligeramente en F1 micro (0,545 frente a 0,554), una métrica dominada por las etiquetas frecuentes. En el rating la ventaja es más clara (MAE 442 frente a 471; Spearman 0,713 frente a 0,681). Muchas etiquetas tienen pistas léxicas directas (*tree*, *string*, *modulo*) que una bolsa de palabras ya captura bien; la ventaja del *encoder* se nota más en la dificultad, que depende de cómo se combinan la descripción y las restricciones. Con 985 problemas de test, diferencias de un punto pueden deberse al azar, así que en etiquetas lo prudente es hablar de rendimiento comparable.
- **B0** obtiene un F1 micro de 0,326 solo por predecir las etiquetas más frecuentes, y su mAP (0,172) corresponde al nivel del azar. Su Spearman es NaN porque predice siempre la misma media. Todos los modelos lo superan con amplitud.
- La ***subset accuracy*** es muy baja en todos los modelos (≤ 0,03): acertar exactamente el conjunto de etiquetas es muy exigente con 20 etiquetas posibles y una media de 3,43 etiquetas por problema en test.

5.4. Efecto del ajuste de umbrales

In [51]:

```
threshold_rows = {}
for name in ROW_ORDER[1:]:
    fixed = evaluate(name, thresholds=0.5)
    tuned = evaluate(name)
    threshold_rows[name] = {"F1 micro (0.5)": fixed["f1_micro"],
                            "F1 micro (ajustado)": tuned["f1_micro"],
                            "F1 macro (0.5)": fixed["f1_macro"],
                            "F1 macro (ajustado)": tuned["f1_macro"]}
pd.DataFrame(threshold_rows).T.round(3)
```

Out[51]:

	F1 micro (0.5)	F1 micro (ajustado)	F1 macro (0.5)	F1 macro (ajustado)
TF-IDF + LR	0.540	0.554	0.515	0.519
Linear probe	0.471	0.483	0.428	0.444
ModernBERT ajustado	0.545	0.545	0.524	0.530

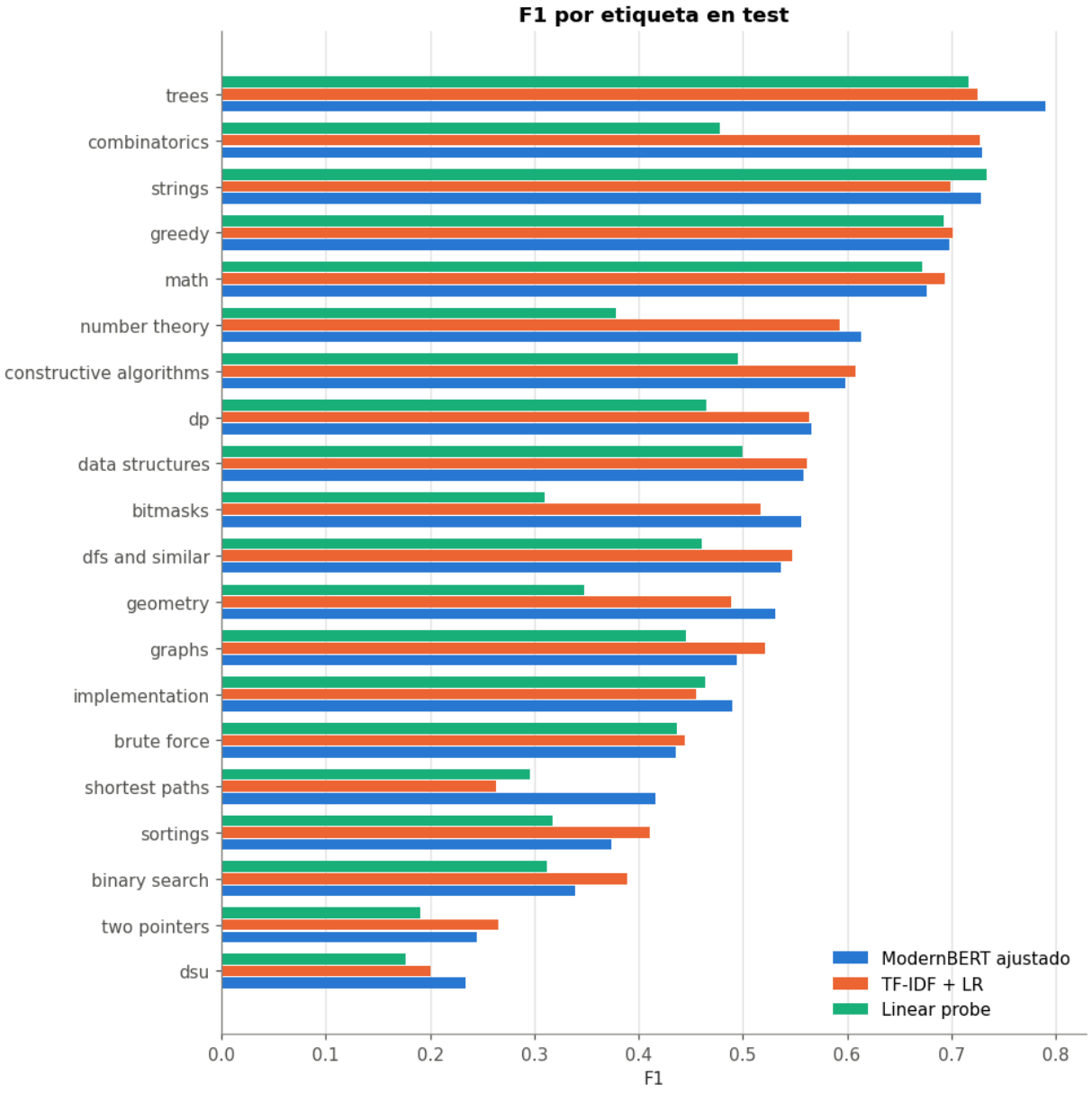
Efecto de los umbrales.

- En TF-IDF y en el *linear probe*, ajustar un umbral por etiqueta mejora el F1 micro algo más de un punto (0,540 → 0,554 y 0,471 → 0,483) y el F1 macro entre 0,4 y 1,6 puntos.
- En el modelo ajustado apenas cambia: el F1 micro se queda en 0,545 y el macro sube de 0,524 a 0,530. La pérdida con `pos_weight` ya desplaza hacia arriba las probabilidades de las etiquetas raras, así que el umbral 0,5 es un punto de partida razonable.
- Los umbrales elegidos en validación (sección 5) están casi todos **por encima de 0,5** (entre 0,55 y 0,8), salvo `implementation` (0,3), `brute force` (0,4) y `two pointers` (0,5). Con `pos_weight`, el modelo da probabilidades altas con facilidad y conviene exigirle más confianza antes de predecir una etiqueta.

5.5. Análisis por etiqueta

In [52]:
```
per_tag = {}
for name in ["ModernBERT ajustado", "TF-IDF + LR", "Linear probe"]:
    res = results[name]
    pred = (res["test_scores"] >= res["thresholds"]).astype(int)
    p, r, f, _ = precision_recall_fscore_support(Y["test"], pred, zero_division=0)
    per_tag[name] = pd.DataFrame({"precisión": p, "recall": r, "F1": f}, index=TAGS)
per_tag_ft = per_tag["ModernBERT ajustado"].copy()
per_tag_ft["frecuencia train"] = Y["train"].sum(axis=0).astype(int)
per_tag_ft["positivos test"] = Y["test"].sum(axis=0).astype(int)

f1_by_model = pd.DataFrame({n: t["F1"] for n, t in per_tag.items()})
f1_by_model = f1_by_model.sort_values("ModernBERT ajustado")
fig, ax = plt.subplots(figsize=(9, 9))
y_pos = np.arange(len(TAGS))
height = 0.26
for i, name in enumerate(f1_by_model.columns):
    ax.barh(y_pos + (i - 1) * height, f1_by_model[name], height=height * 0.9,
            color=MODEL_COLORS[name], label=name)
ax.set_yticks(y_pos, f1_by_model.index)
ax.set(title="F1 por etiqueta en test", xlabel="F1")
ax.grid(axis="y", visible=False)
ax.legend(frameon=False, loc="lower right")
plt.tight_layout()
plt.show()
per_tag_ft.sort_values("F1", ascending=False).round(3)
```



Out[52]:

	precisión	recall	F1	frecuencia train	positivos test
trees	0.846	0.740	0.790	619	104
combinatorics	0.755	0.706	0.730	498	109
strings	0.743	0.714	0.728	579	77
greedy	0.636	0.773	0.698	2137	463
math	0.656	0.698	0.677	2194	421
number theory	0.761	0.514	0.614	563	105
constructive algorithms	0.575	0.624	0.598	1308	234
dp	0.561	0.571	0.566	1639	282
data structures	0.530	0.591	0.559	1356	242
bitmasks	0.689	0.467	0.556	412	90
dfs and similar	0.500	0.578	0.536	723	116
geometry	0.591	0.481	0.531	329	27
graphs	0.457	0.536	0.494	865	110
implementation	0.363	0.753	0.490	2252	288
brute force	0.304	0.764	0.435	1252	275
shortest paths	0.400	0.435	0.417	221	23
sortings	0.309	0.474	0.374	826	135
binary search	0.311	0.373	0.339	796	158
two pointers	0.154	0.595	0.244	403	74
dsu	0.239	0.229	0.234	274	48

Análisis por etiqueta.

- Las etiquetas que mejor predice el modelo son `trees` (F1 0,790), `combinatorics` (0,730), `strings` (0,728), `greedy` (0,698) y `math` (0,677). Salvo `greedy`, todas tienen un vocabulario muy reconocible en el enunciado: árboles, cadenas o conteos módulo un primo.
- Las peores son `dsu` (0,234), `two pointers` (0,244), `binary search` (0,339), `sortings` (0,374) y `shortest paths` (0,417). Son técnicas de solución que el enunciado rara vez menciona y que a menudo son **alternativas** a otras: en la tabla de coocurrencia de la sección 3.6, `dsu` aparece con `dfs and similar` con un *lift* de 5,13, y `two pointers` con `binary search` con un *lift* de 3,83. Varias técnicas resuelven el mismo problema, y el modelo no puede saber cuál eligieron quienes etiquetaron.
- **La frecuencia no lo explica todo**: `implementation` es la etiqueta más frecuente en train (2252 problemas) y obtiene solo 0,490, mientras que `trees` (619) es la mejor. `implementation`, `brute force` y `two pointers` comparten un patrón: *recall* alto (0,753; 0,764; 0,595) y precisión muy baja (0,363; 0,304; 0,154). El modelo las usa como "comodín" cuando el enunciado no da pistas claras.
- Frente a los baselines (figura), el modelo ajustado destaca sobre todo en `trees`, `shortest paths`, `geometry`, `bitmasks` y `number theory`, mientras que TF-IDF es algo mejor en `graphs`, `sortings` y `binary search`. Algunas etiquetas tienen muy pocos positivos en test (23 en `shortest paths`, 27 en `geometry`), así que sus diferencias son poco fiables.

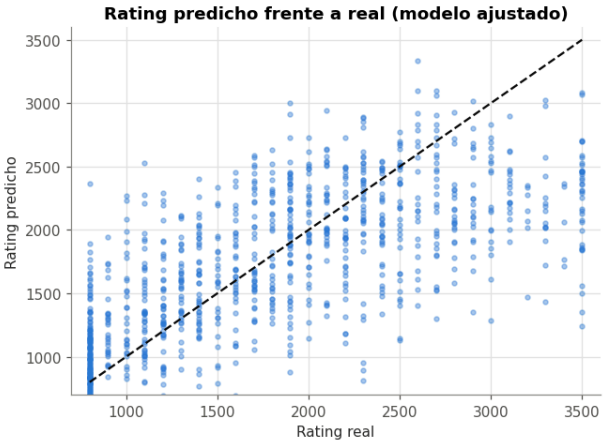
5.6. Predicción del rating

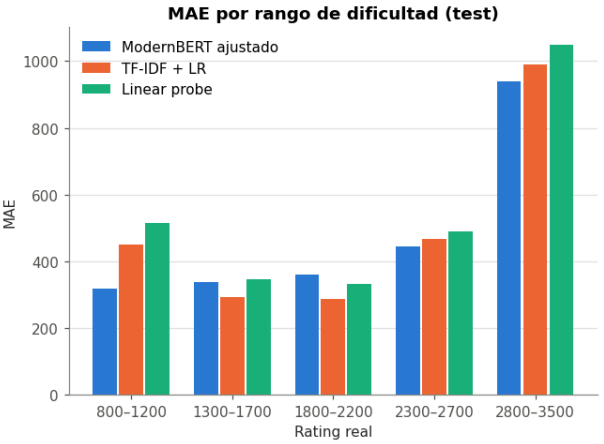
In [53]:

```
mask_test = M["test"].astype(bool)
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
ax = axes[0]
ax.scatter(R["test"][mask_test], ft_test_rating[mask_test], s=10, alpha=0.4,
           color=COLORS["train"])
ax.plot([800, 3500], [800, 3500], color="#0b0b0b", linestyle="--", linewidth=1.5)
ax.set(title="Rating predicho frente a real (modelo ajustado)", xlabel="Rating real",
       ylabel="Rating predicho", xlim=(700, 3600), ylim=(700, 3600))

buckets = [(800, 1200), (1300, 1700), (1800, 2200), (2300, 2700), (2800, 3500)]
bucket_rows = {}
for name in ["ModernBERT ajustado", "TF-IDF + LR", "Linear probe"]:
    err = np.abs(results[name]["test_rating"] - R["test"])
    bucket_rows[name] = [err[mask_test & (R["test"] >= lo) & (R["test"] <= hi)].mean()
                         for lo, hi in buckets]
bucket_df = pd.DataFrame(bucket_rows, index=[f"{lo}–{hi}" for lo, hi in buckets])
x = np.arange(len(buckets))
for i, name in enumerate(bucket_df.columns):
    axes[1].bar(x + (i - 1) * 0.26, bucket_df[name], width=0.24, color=MODEL_COLORS[name],
                label=name)
axes[1].set_xticks(x, bucket_df.index)
axes[1].set(title="MAE por rango de dificultad (test)", xlabel="Rating real", ylabel="MAE")
axes[1].grid(axis="x", visible=False)
axes[1].legend(frameon=False)
plt.tight_layout()
plt.show()
```

```
bucket_df.assign(problemas=[int((mask_test & (R["test"] >= lo) & (R["test"] <= hi)).sum())
                            for lo, hi in buckets]).round(1)
```





Out[53]:

	ModernBERT ajustado	TF-IDF + LR	Linear probe	problemas
800–1200	318.0	450.7	514.6	299
1300–1700	337.9	293.4	344.9	182
1800–2200	359.5	287.3	333.3	195
2300–2700	445.4	466.1	489.1	157
2800–3500	939.1	990.0	1049.2	144

Análisis del rating.

- La nube de puntos queda por encima de la diagonal en los problemas fáciles y por debajo en los difíciles: el modelo tiende a predecir valores intermedios (**regresión a la media**). El error cuadrático penaliza mucho los errores grandes y, ante la duda, compensa acercarse a la media.
- Por rangos, el modelo ajustado es el mejor **en los extremos**: en 800–1200 su MAE es 318, frente a 451 de TF-IDF y 515 del *linear probe*, y en 2800–3500 es 939, frente a 990 y 1049. En los rangos centrales TF-IDF es algo mejor (293 frente a 338 en 1300–1700 y 287 frente a 360 en 1800–2200).
- El error se dispara en los problemas más difíciles (MAE 939 en 2800–3500, con 144 problemas). Por encima de 2800, el enunciado no suele dar más pistas que el de un problema de 2200: la dificultad real depende de la idea de la solución, no solo del texto.

5.7. Análisis de errores

Ejemplos de aciertos y fallos del modelo ajustado en test, ordenados por el F1 de cada problema.

In [54]:
```
pred_test = (ft_test_probs >= ft_thresholds).astype(int)
sample_f1 = f1_score(Y["test"].T, pred_test.T, average=None, zero_division=0)


def show_examples(indices):
    rows = []
    for i in indices:
        row = data["test"].iloc[i]
        statement = row["text"].split("Statement: ", 1)[-1]
        rows.append({
            "título": row["title"],
            "enunciado (fragmento)": statement[:300] + "...",
            "etiquetas reales": ", ".join(row["tags_kept"]),
            "etiquetas predichas": ", ".join(TAGS[k] for k in np.flatnonzero(pred_test[i])),
            "F1": round(sample_f1[i], 2),
            "rating real": row["rating"],
            "rating predicho": round(ft_test_rating[i]),
        })
    return pd.DataFrame(rows)


ranked = np.argsort(-sample_f1, kind="stable")
with pd.option_context("display.max_colwidth", 400):
    print("Aciertos (mayor F1 por problema)")
    display(show_examples(ranked[:3]))
    print("Fallos (menor F1 por problema)")
    display(show_examples(ranked[-3:]))
```

```
Aciertos (mayor F1 por problema)
```

	título	enunciado (fragmento)	etiquetas reales	etiquetas predichas	F1	rating real	rating predicho
0	Short Permutation Problem	You are given an integer n. For each (m, k) such that 3 <= m <= n+1 and 0 <= k <= n-1, count the permutations of [1, 2, ..., n] such that p_i + p_{i+1} >= m for exactly k indices i, modulo 998\,244\,353.\nNote: In the first test case, the answers for all (m, k) are shown in the following table: k = 0...	combinatorics, dp, math	math, dp, combinatorics	1.0	1900.0	2731
1	Balanced Subsequences	A sequence of brackets is called balanced if one can turn it into a valid math expression by adding characters '+' and '1'. For example, sequences '(())()', '()', and '((()(()))' are balanced, while ')(', '(()', and '(()))(' are not. A subsequence is a sequence that can be derived from the given sequ...	combinatorics, dp, math	math, dp, combinatorics	1.0	2700.0	2146
2	A Balanced Problemset?	Jay managed to create a problem of difficulty x and decided to make it the second problem for Codeforces Round #921. But Yash fears that this problem will make the contest highly unbalanced, and the coordinator will reject it. So, he decided to break it up into a problemset of n sub-problems such th...	brute force, greedy, math, number theory	math, greedy, brute force, number theory	1.0	1200.0	1151

Fallos (menor F1 por problema)

	título	enunciado (fragmento)	etiquetas reales	etiquetas predichas	F1	rating real	rating predicho
0	Kevin and Numbers	Kevin wrote an integer sequence a of length n on the blackboard. Kevin can perform the following operation any number of times: - Select two integers x, y on the blackboard such that |x - y| <= 1, erase them, and then write down an integer x + y instead. Kevin wants to know if it is possible to tran...	bitmasks, data structures	math, greedy, constructive algorithms, brute force, two pointers	0.0	1600.0	1089
1	Kevin and Nivek	Kevin and Nivek are competing for the title of "The Best Kevin". They aim to determine the winner through n matches. The i-th match can be one of two types: - Type 1: Kevin needs to spend a_i time to defeat Nivek and win the match. If Kevin doesn't spend a_i time on it, Nivek will win the match. - T...	dp	implementation, greedy, sortings, binary search, two pointers	0.0	3500.0	1244
2	Permutation Factory	You are given two permutations p_1,p_2,...,p_n and q_1,q_2,...,q_n of length n. In one operation, you can select two integers 1<= i,j<= n,i!= j and swap p_i and p_j. The cost of the operation is min (|i-j|,|p_i-p_j|). Find the minimum cost to make p_i = q_i hold for all 1<= i<= n and output a sequen...	geometry, graphs	implementation, greedy, constructive algorithms, brute force	0.0	3500.0	1866

Patrones en los errores.

- Los aciertos completos son problemas con vocabulario explícito. Por ejemplo, *Short Permutation Problem* pide contar permutaciones "modulo 998 244 353", una pista directa de combinatoria con programación dinámica. Aun así, el rating puede fallar mucho aunque las etiquetas sean perfectas (1900 real frente a 2731 predicho).
- Los fallos totales (F1 = 0) son problemas cuya técnica no se deduce del enunciado. *Kevin and Nivek* es un problema de `dp` de rating 3500 que el modelo etiqueta con `implementation`, `greedy`, `sortings`, `binary search` y `two pointers`; las etiquetas de *Permutation Factory* (`geometry`, `graphs`, rating 3500) dependen de cómo se modela la solución, no de lo que cuenta el texto.
- En los fallos, el modelo recurre a las etiquetas genéricas y frecuentes (`implementation`, `greedy`, `constructive algorithms`, `brute force`), justo las de *recall* alto y precisión baja del análisis por etiqueta, y **subestima el rating** de los problemas más difíciles (3500 → 1244 y 3500 → 1866).
- El ejemplo de uso del modelo publicado muestra el mismo sesgo: en un problema fácil de `implementation` (rating 1100) acierta esa etiqueta, pero añade otras cuatro.

5.8. Limitaciones

- **Ruido en las etiquetas.** Las etiquetas las asignan los autores y la comunidad, y un problema puede admitir varias soluciones válidas con técnicas distintas: parte del "error" no es tal.
- **Desbalance.** Las etiquetas raras tienen pocos ejemplos; `pos_weight` y los umbrales por etiqueta lo mitigan, pero no lo eliminan.
- **Dificultad intrínseca del rating.** El rating depende de cómo se comportaron los participantes en el concurso, algo que el enunciado no refleja del todo.
- **Truncamiento.** Los textos más largos que `MAX_LEN` pierden su parte final.
- **Deriva temporal de las etiquetas.** Los problemas recientes llevan más etiquetas (2,54 por problema en train, 3,15 en validación y 3,43 en test) y algunas ganan mucho peso: `greedy` pasa del 28,2 % de los problemas de train al 47,0 % en test. El modelo aprende de problemas etiquetados de forma más escueta que aquellos con los que se evalúa; los umbrales ajustados en validación lo compensan solo en parte.
- **Solo inglés** y solo problemas de Codeforces.

6. Conclusiones

6.1. Resumen de resultados

- Se ha ajustado **ModernBERT-base completo**, con un modelo propio de dos cabezas, para predecir a la vez las etiquetas algorítmicas y el rating de problemas de Codeforces a partir del enunciado. El entrenamiento tarda 35,5 minutos en una T4.
- En test (985 problemas, los más recientes), el modelo ajustado obtiene un **F1 macro de 0,530**, un **mAP de 0,528** y un **MAE de 442 puntos** de rating, con una correlación de Spearman de 0,713.
- **El ajuste fino aporta claramente** frente al modelo base sin ajustar: respecto al *linear probe*, +0,086 de F1 macro, +0,120 de mAP y 79 puntos menos de MAE.
- Frente a un baseline TF-IDF bien ajustado, el rendimiento en etiquetas es **comparable** (mejor en F1 macro y mAP, algo peor en F1 micro) y en el rating es **mejor en todas las métricas**. En esta tarea las pistas léxicas pesan mucho, y con 7585 problemas de entrenamiento la ventaja de un *encoder* moderno es moderada.
- Las etiquetas con vocabulario propio (`trees` , `strings` , `combinatorics`) se predicen bien; las que dependen de la idea de la solución o son intercambiables con otras (`dsu` , `two pointers` , `binary search`) son las más difíciles. El rating se predice mejor en el rango medio que en los extremos, sobre todo en los problemas más difíciles, por regresión a la media.

6.2. Diferencias con los notebooks de clase

	Notebook 6 (BERT)	Notebook 7 (QLoRA)	Este proyecto
Tarea	Análisis de sentimiento (Yelp, 1–5 estrellas)	Seguimiento de instrucciones (Guanaco)	Etiquetas algorítmicas + dificultad de problemas de Codeforces
Tipo de salida	Una clase de 5 (multiclase)	Texto libre	Multi-etiqueta (20 etiquetas) + regresión
Modelo	BERT-base-cased y RoBERTa-base	Qwen2.5-0.5B (base)	ModernBERT-base
Técnica de ajuste	Ajuste fino completo	QLoRA (4 bits + LoRA r=16) con `SFTTrainer`	Ajuste fino completo con `Trainer` y modelo propio
Cabeza y pérdida	`AutoModelForSequenceClassification` , entropía cruzada	Cabeza de LM, pérdida de LM	Cabeza doble propia; BCE ponderada + MSE enmascarado
Longitud de contexto	`padding="max_length"` (512)	1024	1024, con padding dinámico y agrupación por longitud
Datos y división	1000 ejemplos aleatorios de train y de test	1000 ejemplos, sin validación	~10 000 problemas, división temporal train/val/test
Umbrales	No aplica (argmax)	No aplica	Un umbral por etiqueta ajustado en validación
Evaluación	*Accuracy*	Comparación cualitativa de respuestas	F1 micro/macro, mAP, MAE, Spearman y 3 baselines

6.3. Aportaciones

- **Clasificación multi-etiqueta** con pesos por clase y **umbrales por etiqueta**, en lugar de la clasificación multiclase con `argmax` del notebook 6.
- **Modelo multitarea propio**: subclases de `PretrainedConfig` y `PreTrainedModel` con dos cabezas y una pérdida combinada, compatibles con `Trainer` , `push_to_hub` y `from_pretrained` .
- **Contexto largo** (1024 tokens) con un *encoder* moderno, padding dinámico y agrupación por longitud.
- **Evaluación cuidadosa**: división temporal sin mezclar concursos, deduplicación, umbrales elegidos solo con validación y comparación con el modelo base sin ajustar (*linear probe*).